Building ZK-STARK

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 The FRI Protocol

2 Arithmetic Intermediate Representation

Where are we?

- Previous lecture: any Poly-IOP becomes an IOP via **bind, quotient, batch**, using a single black box: a δ-RS-IOPP that tests whether $h : \Omega \to \mathbb{F}$ is close to RS$[\mathbb{F}, \Omega, k]$.

Where are we?

- Previous lecture: any Poly-IOP becomes an IOP via **bind, quotient, batch**, using a single black box: a δ-RS-IOPP that tests whether $h : \Omega \to \mathbb{F}$ is close to $\mathrm{RS}[\mathbb{F}, \Omega, k]$.
- Today, part I: open this box — the **FRI** protocol.

Where are we?

- Previous lecture: any Poly-IOP becomes an IOP via **bind, quotient, batch**, using a single black box: a δ-RS-IOPP that tests whether $h : \Omega \to \mathbb{F}$ is close to RS$[\mathbb{F}, \Omega, k]$.
- Today, part I: open this box — the **FRI** protocol.
- Today, part II: the Poly-IOP itself — the **Arithmetic Intermediate Representation** (AIR).

Where are we?

- Previous lecture: any Poly-IOP becomes an IOP via **bind, quotient, batch**, using a single black box: a δ-RS-IOPP that tests whether $h : \Omega \to \mathbb{F}$ is close to RS$[\mathbb{F}, \Omega, k]$.
- Today, part I: open this box — the **FRI** protocol.
- Today, part II: the Poly-IOP itself — the **Arithmetic Intermediate Representation** (AIR).
- Together: a **STARK** (Scalable Transparent ARgument of Knowledge), which needs only a hash function: no trusted setup, plausibly post-quantum.

The FRI Protocol

Why is the Problem Non-trivial?

- Without the prover's help, the verifier needs $k + 1$ queries: interpolate h through k points and check one more point.

Why is the Problem Non-trivial?

- Without the prover's help, the verifier needs $k + 1$ queries: interpolate h through k points and check one more point.
- In applications, k is the size of the whole computation: as slow as recomputing everything.

Why is the Problem Non-trivial?

- Without the prover's help, the verifier needs $k + 1$ queries: interpolate h through k points and check one more point.
- In applications, k is the size of the whole computation: as slow as recomputing everything.
- **FRI** (Fast Reed–Solomon IOP of Proximity): the prover sends a few additional oracles, after which the verifier needs only $O(\log n)$ queries.

Why is the Problem Non-trivial?

- Without the prover's help, the verifier needs $k + 1$ queries: interpolate h through k points and check one more point.
- In applications, k is the size of the whole computation: as slow as recomputing everything.
- **FRI** (Fast Reed–Solomon IOP of Proximity): the prover sends a few additional oracles, after which the verifier needs only $O(\log n)$ queries.

Main ingredient: folding

Folding maps "is g close to RS$[\mathbb{F}, \Omega, k]$?" to the same problem for Fold(g, β): half the dimension, half the domain. After $\log_2 k$ folds the code consists of **constants**, and the prover simply sends the constant.

We assume $k = 2^\ell$ and denote the folding challenges by β.

Recall: 2-way Folding

Definition (2-way folding)

For $g : \Omega \to \mathbb{F}$ and $\beta \in \mathbb{F}$, define $g_E, g_O : \Omega^2 \to \mathbb{F}$ by

$$g_E(x^2) := \frac{g(x) + g(-x)}{2}, \qquad g_O(x^2) := \frac{g(x) - g(-x)}{2x},$$

and $\text{Fold}(g, \beta) := g_E + \beta \cdot g_O : \Omega^2 \to \mathbb{F}$.

Recall: 2-way Folding

Definition (2-way folding)

For $g : \Omega \to \mathbb{F}$ and $\beta \in \mathbb{F}$, define $g_E, g_O : \Omega^2 \to \mathbb{F}$ by

$$g_E(x^2) := \frac{g(x) + g(-x)}{2}, \qquad g_O(x^2) := \frac{g(x) - g(-x)}{2x},$$

and $\mathrm{Fold}(g, \beta) := g_E + \beta \cdot g_O : \Omega^2 \to \mathbb{F}$.

Obstacle

The verifier cannot compute $\mathrm{Fold}(g, \beta)$ itself: that requires reading all of g.

Recall: 2-way Folding

Definition (2-way folding)

For $g : \Omega \to \mathbb{F}$ and $\beta \in \mathbb{F}$, define $g_E, g_O : \Omega^2 \to \mathbb{F}$ by

$$g_E(x^2) := \frac{g(x) + g(-x)}{2}, \qquad g_O(x^2) := \frac{g(x) - g(-x)}{2x},$$

and $\mathrm{Fold}(g, \beta) := g_E + \beta \cdot g_O : \Omega^2 \to \mathbb{F}$.

Obstacle

The verifier cannot compute $\mathrm{Fold}(g, \beta)$ itself: that requires reading all of g.

Solution: the prover sends the fold as a new oracle, and the verifier spot-checks it. Folding is **local**: $\mathrm{Fold}(g, \beta)(x^2)$ depends only on $g(x)$ and $g(-x)$, so one value of the fold costs two cells of g.

Folding Geometrically

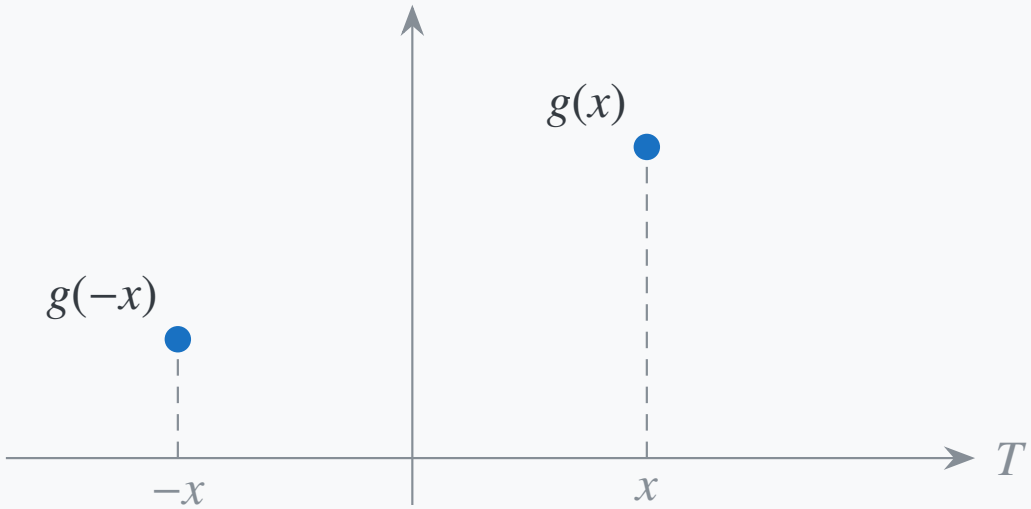


Figure: The values of g at $\pm x$ determine a line L; the fold at x^2 is $L(\beta)$. Drawn over the reals only for intuition.

Folding Geometrically

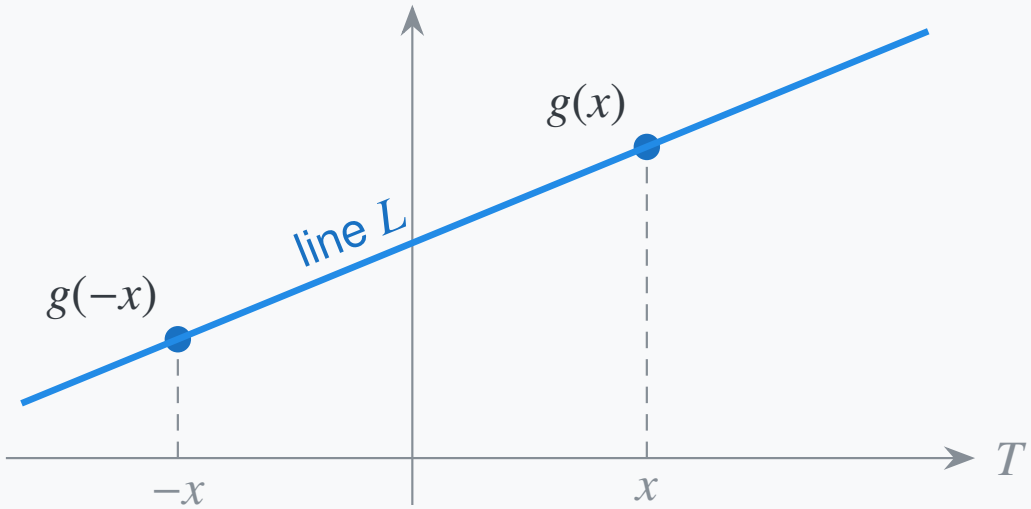


Figure: The values of g at $\pm x$ determine a line L; the fold at x^2 is $L(\beta)$. Drawn over the reals only for intuition.

Folding Geometrically

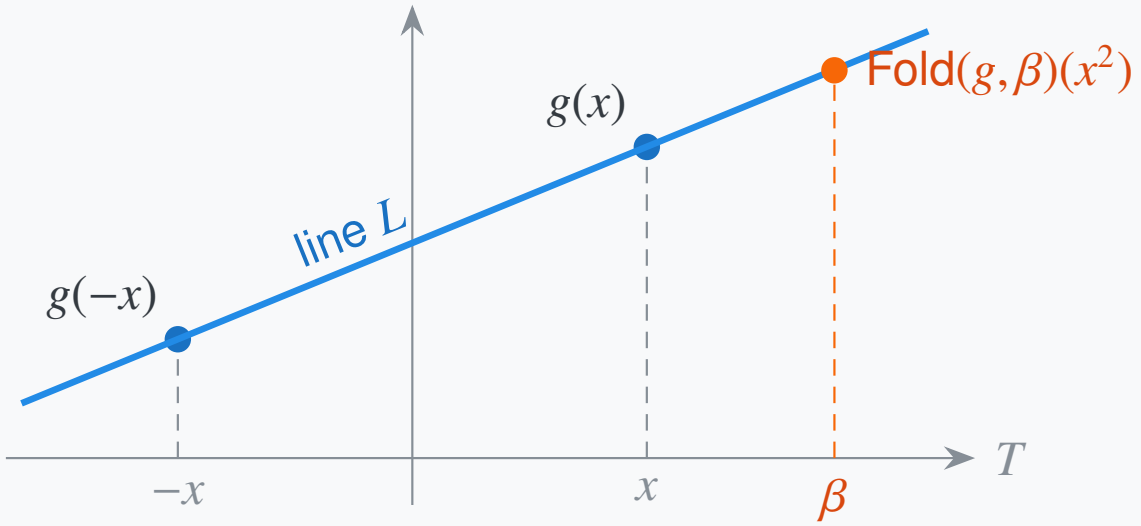


Figure: The values of g at $\pm x$ determine a line L; the fold at x^2 is $L(\beta)$. Drawn over the reals only for intuition.

Folding Geometrically

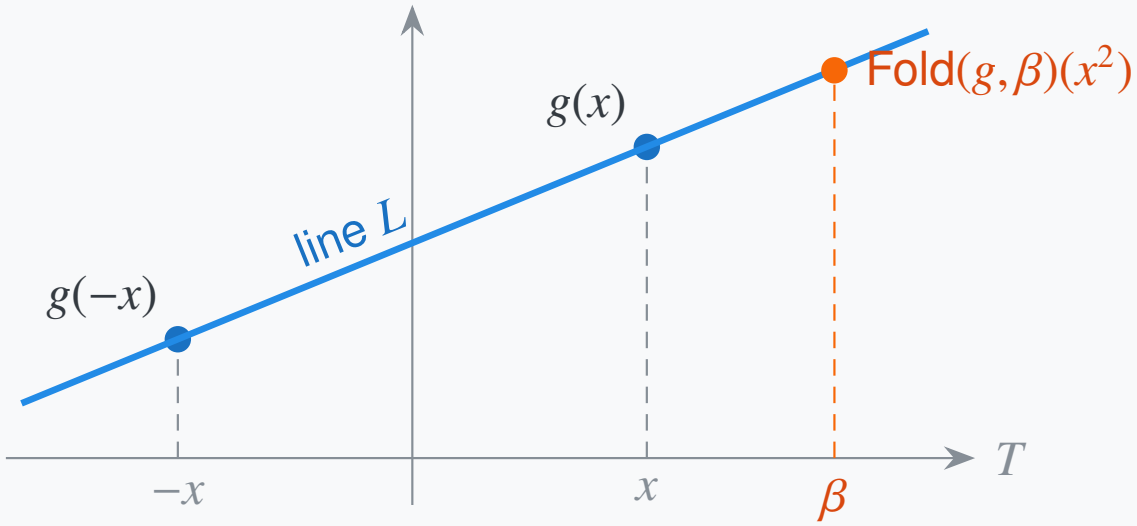


Figure: The values of g at $\pm x$ determine a line L; the fold at x^2 is $L(\beta)$. Drawn over the reals only for intuition.

Question #1

Let L be the unique polynomial of degree at most 1 with $L(x) = g(x)$ and $L(-x) = g(-x)$. Show that $\text{Fold}(g,\beta)(x^2) = L(\beta)$.

Folding: Example

Example

$\mathbb{F} = \mathbb{F}_{17}$, $\Omega = \langle 2 \rangle$ listed as $1, 2, 4, 8, 16, 15, 13, 9$, $k = 4$ ($\rho = 1/2$). The polynomial $f = 3 + T + 2T^2 + 5T^3$ gives

$$g = \bar{f} = (11, 2, 2, 13, 16, 3, 0, 11).$$

Folding: Example

Example

$\mathbb{F} = \mathbb{F}_{17}$, $\Omega = \langle 2 \rangle$ listed as $1, 2, 4, 8, 16, 15, 13, 9$, $k = 4$ ($\rho = 1/2$). The polynomial $f = 3 + T + 2T^2 + 5T^3$ gives

$$g = \bar{f} = (11, 2, 2, 13, 16, 3, 0, 11).$$

Take $\beta = 2$. The fold at $1^2 = 1$ uses the pair $\{1, 16\} = \{1, -1\}$:

$$\text{Fold}(g, 2)(1) = \frac{11 + 16}{2} + 2 \cdot \frac{11 - 16}{2} = 27 \cdot 9 - 5 = 0 \pmod{17},$$

using $2^{-1} = 9$ in $\mathbb{F}_{17}$.

Folding: Example

Example

$\mathbb{F} = \mathbb{F}_{17}$, $\Omega = \langle 2 \rangle$ listed as $1, 2, 4, 8, 16, 15, 13, 9$, $k = 4$ ($\rho = 1/2$). The polynomial $f = 3 + T + 2T^2 + 5T^3$ gives

$$g = \bar{f} = (11, 2, 2, 13, 16, 3, 0, 11).$$

Take $\beta = 2$. The fold at $1^2 = 1$ uses the pair $\{1, 16\} = \{1, -1\}$:

$$\text{Fold}(g, 2)(1) = \frac{11 + 16}{2} + 2 \cdot \frac{11 - 16}{2} = 27 \cdot 9 - 5 = 0 \pmod{17},$$

using $2^{-1} = 9$ in $\mathbb{F}_{17}$.

The other three pairs give $\text{Fold}(g, 2) = (0, 2, 10, 8)$ on Ω^2 (listed as $1, 4, 16, 13$): the evaluations of $f_E + 2f_O = 5 + 12T$, of degree less than 2. ✓

Notation

- $\Omega_i := \Omega^{2^i}$ is the domain after i folds: $\Omega_0 = \Omega$, $\Omega_1 = \Omega^2$, ...

Notation

- $\Omega_i := \Omega^{2^i}$ is the domain after i folds: $\Omega_0 = \Omega$, $\Omega_1 = \Omega^2$, ...
- $C_i := \mathsf{RS}[\mathbb{F}, \Omega_i, k/2^i]$ is the corresponding code.

Notation

- $\Omega_i := \Omega^{2^i}$ is the domain after i folds: $\Omega_0 = \Omega$, $\Omega_1 = \Omega^2$, ...
- $C_i := \mathsf{RS}[\mathbb{F}, \Omega_i, k/2^i]$ is the corresponding code.
- All these codes have the **same rate** ρ, and the last one, C_ℓ, consists of the constant functions.

Notation

- $\Omega_i := \Omega^{2^i}$ is the domain after i folds: $\Omega_0 = \Omega$, $\Omega_1 = \Omega^2$, ...
- $C_i := \mathsf{RS}[\mathbb{F}, \Omega_i, k/2^i]$ is the corresponding code.
- All these codes have the **same rate** ρ, and the last one, C_ℓ, consists of the constant functions.

Two phases:

- **commit phase**: the prover sends every fold as a new oracle;
- **query phase**: only afterwards, the verifier spot-checks consecutive oracles.

The FRI Protocol

Definition (FRI)

Input: an oracle $g_0 := g : \Omega_0 \to \mathbb{F}$; goal: test proximity to $C_0 = \mathsf{RS}[\mathbb{F}, \Omega, k]$.

The FRI Protocol

Definition (FRI)

Input: an oracle $g_0 := g : \Omega_0 \to \mathbb{F}$; goal: test proximity to $C_0 = \mathsf{RS}[\mathbb{F}, \Omega, k]$.

- **Commit phase.** For every $i \in [\ell]$:
  1: the verifier samples $\beta_i \leftarrow\!\$\ \mathbb{F}$;
  2: the prover sends $g_{i+1} : \Omega_{i+1} \to \mathbb{F}$; honestly, $g_{i+1} := \mathsf{Fold}(g_i, \beta_i)$.

  Finally, the prover sends a constant $c \in \mathbb{F}$, and we set $g_\ell(y) := c$ for all $y \in \Omega_\ell$.

The FRI Protocol

Definition (FRI)

Input: an oracle $g_0 := g : \Omega_0 \to \mathbb{F}$; goal: test proximity to $C_0 = \mathsf{RS}[\mathbb{F}, \Omega, k]$.

- **Commit phase.** For every $i \in [\ell]$:
  1: the verifier samples $\beta_i \leftarrow\$ \mathbb{F}$;
  2: the prover sends $g_{i+1} : \Omega_{i+1} \to \mathbb{F}$; honestly, $g_{i+1} := \mathsf{Fold}(g_i, \beta_i)$.

  Finally, the prover sends a constant $c \in \mathbb{F}$, and we set $g_\ell(y) := c$ for all $y \in \Omega_\ell$.

- **Query phase.** Repeat s times:
  1: the verifier samples $x_0 \leftarrow\$ \Omega_0$ and sets $x_{i+1} := x_i^2$;
  2: for every $i \in [\ell]$, it queries $g_i(\pm x_i)$ and checks the **round consistency**

$$g_{i+1}(x_{i+1}) = \frac{g_i(x_i) + g_i(-x_i)}{2} + \beta_i \cdot \frac{g_i(x_i) - g_i(-x_i)}{2x_i}.$$

The verifier accepts if and only if all checks pass.

The FRI Protocol

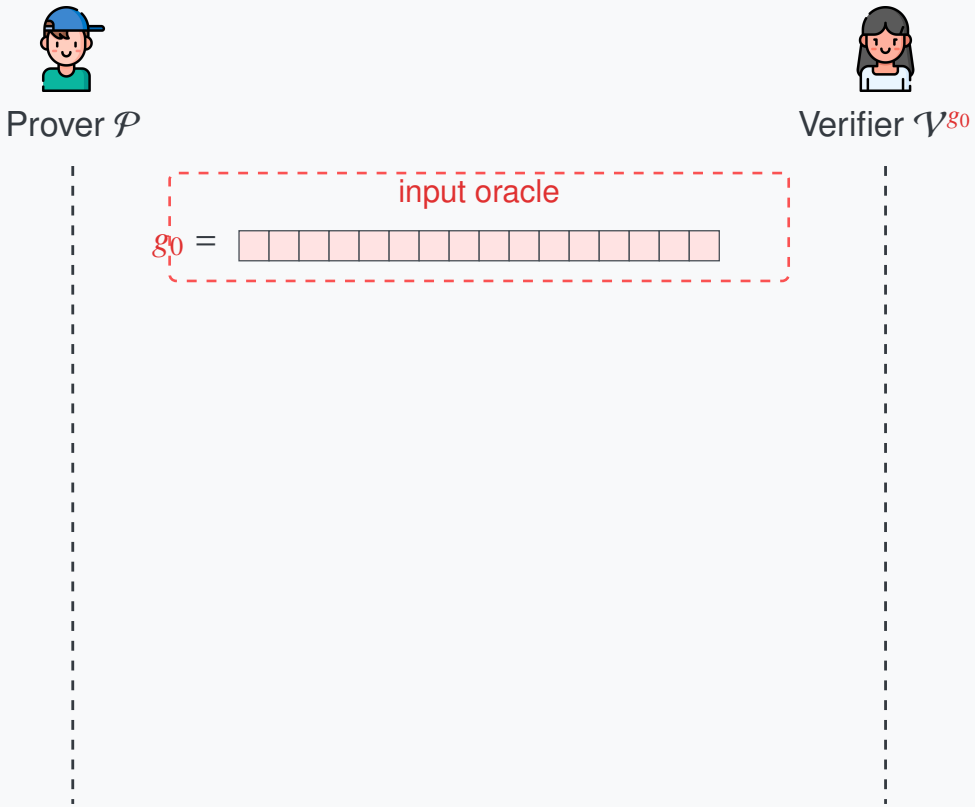


Figure: Each β_i is answered with g_{i+1}, half as long as g_i. A query follows a random point down the layers and reads the pairs $\pm x_i$ (orange).

The FRI Protocol

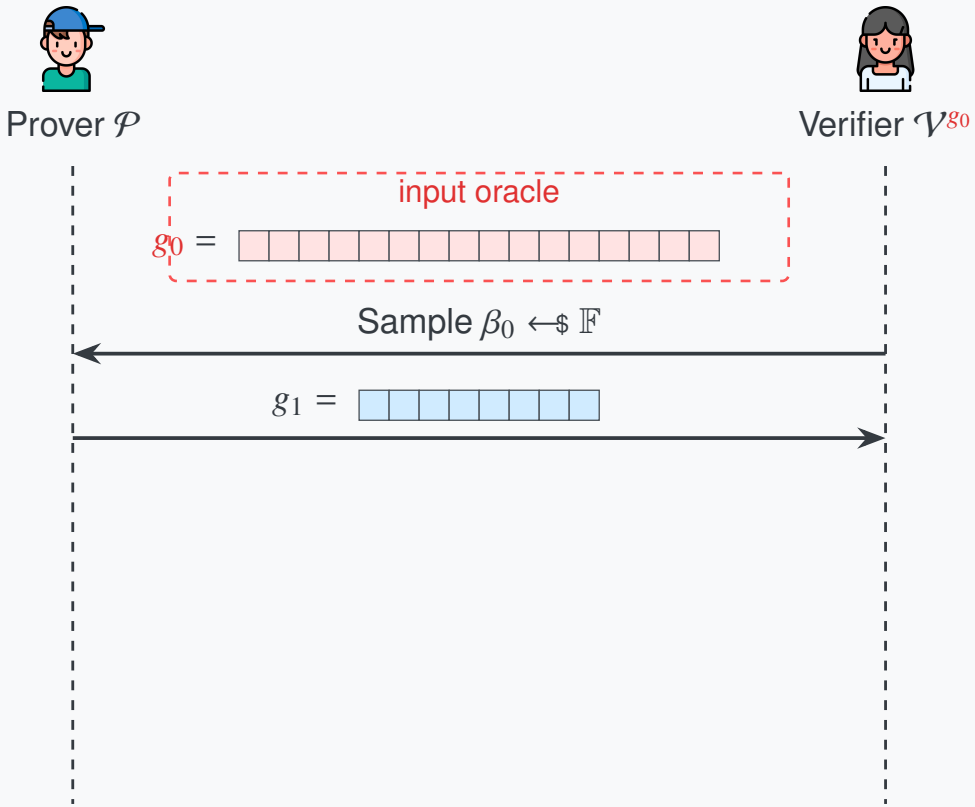


Figure: Each β_i is answered with g_{i+1}, half as long as g_i. A query follows a random point down the layers and reads the pairs $\pm x_i$ (orange).

The FRI Protocol

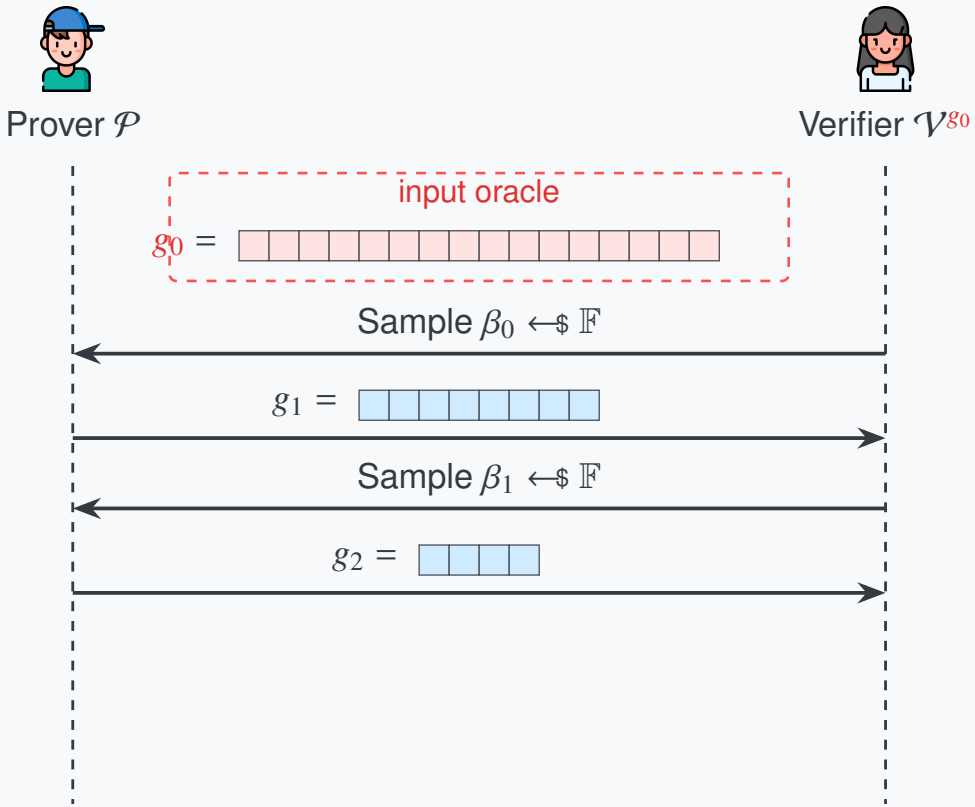


Figure: Each β_i is answered with g_{i+1}, half as long as g_i. A query follows a random point down the layers and reads the pairs $\pm x_i$ (orange).

The FRI Protocol

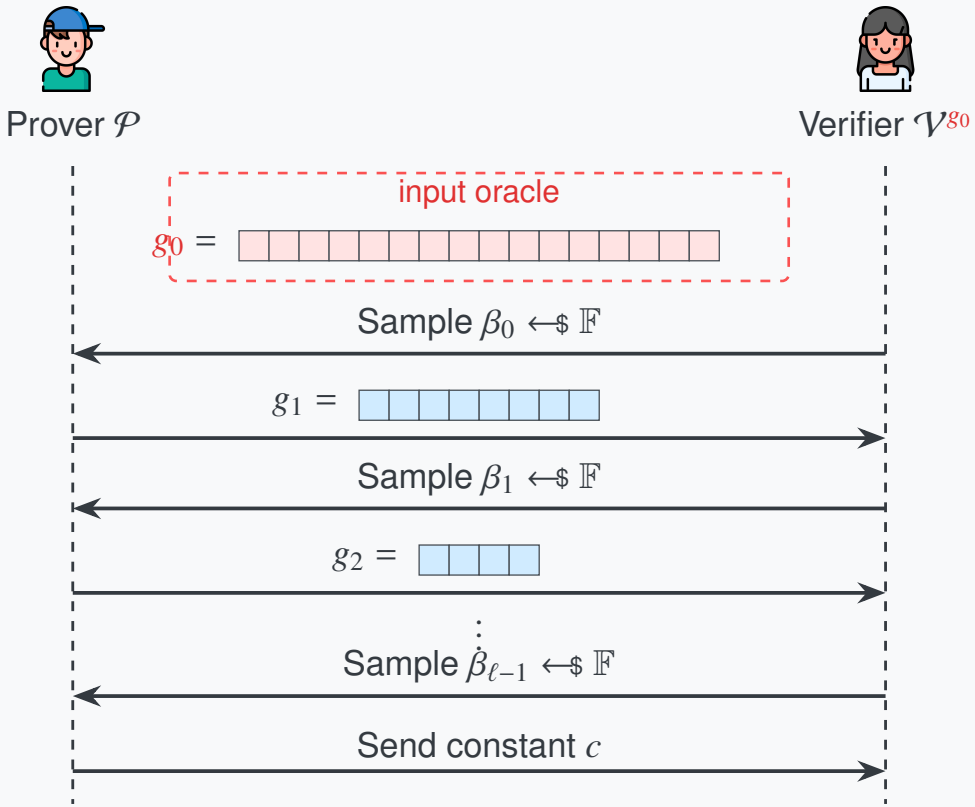


Figure: Each β_i is answered with g_{i+1}, half as long as g_i. A query follows a random point down the layers and reads the pairs $\pm x_i$ (orange).

The FRI Protocol

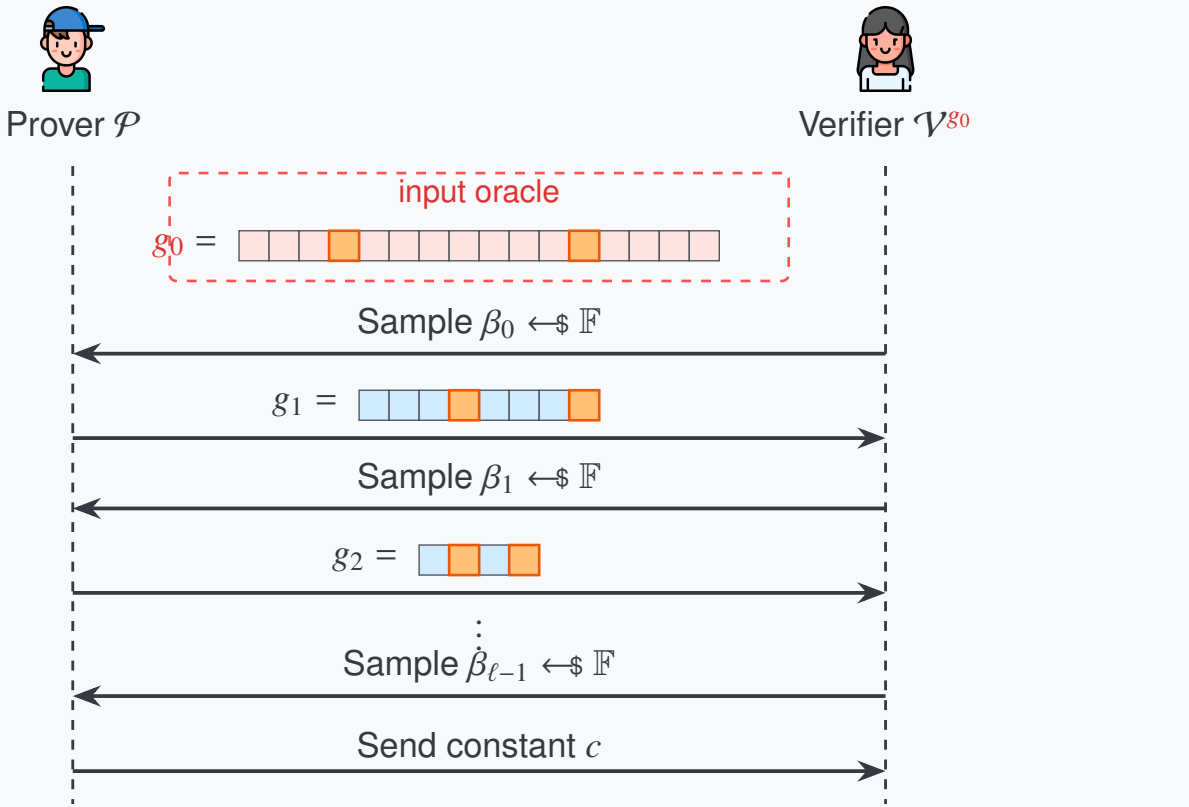


Figure: Each β_i is answered with g_{i+1}, half as long as g_i. A query follows a random point down the layers and reads the pairs $\pm x_i$ (orange).

The FRI Protocol: Observations

- The left-hand side of the round consistency check needs no extra query: $g_{i+1}(x_{i+1})$ is one of the two values queried in the next round; in the last round it is the constant c.

The FRI Protocol: Observations

- The left-hand side of the round consistency check needs no extra query: $g_{i+1}(x_{i+1})$ is one of the two values queried in the next round; in the last round it is the constant c.
- **Completeness:** by the folding lemma, every honest g_i is a codeword of C_i, so g_ℓ is indeed constant.

The FRI Protocol: Example

Example

Continue the folding example with $k = 4$, so FRI has $\ell = 2$ rounds.

The FRI Protocol: Example

Example

Continue the folding example with $k = 4$, so FRI has $\ell = 2$ rounds.

- After $\beta_0 = 2$: $g_1 = (0, 2, 10, 8)$ on $\Omega_1 = \{1, 4, 16, 13\}$.

The FRI Protocol: Example

Example

Continue the folding example with $k = 4$, so FRI has $\ell = 2$ rounds.

- After $\beta_0 = 2$: $g_1 = (0, 2, 10, 8)$ on $\Omega_1 = \{1, 4, 16, 13\}$.
- After $\beta_1 = 5$: $g_2 = (14, 14)$ on $\Omega_2 = \{1, 16\}$, indeed constant, so $c = 14$.

The FRI Protocol: Example

Example

Continue the folding example with $k = 4$, so FRI has $\ell = 2$ rounds.

- After $\beta_0 = 2$: $g_1 = (0, 2, 10, 8)$ on $\Omega_1 = \{1, 4, 16, 13\}$.
- After $\beta_1 = 5$: $g_2 = (14, 14)$ on $\Omega_2 = \{1, 16\}$, indeed constant, so $c = 14$.
- If $x_0 = 2$, then $x_1 = 4$ and the verifier checks

$$g_1(4) = \frac{g_0(2) + g_0(15)}{2} + 2 \cdot \frac{g_0(2) - g_0(15)}{2 \cdot 2} = \frac{2+3}{2} + \frac{2-3}{2} = 2,$$

The FRI Protocol: Example

Example

Continue the folding example with $k = 4$, so FRI has $\ell = 2$ rounds.

- After $\beta_0 = 2$: $g_1 = (0, 2, 10, 8)$ on $\Omega_1 = \{1, 4, 16, 13\}$.
- After $\beta_1 = 5$: $g_2 = (14, 14)$ on $\Omega_2 = \{1, 16\}$, indeed constant, so $c = 14$.
- If $x_0 = 2$, then $x_1 = 4$ and the verifier checks

$$g_1(4) = \frac{g_0(2) + g_0(15)}{2} + 2 \cdot \frac{g_0(2) - g_0(15)}{2 \cdot 2} = \frac{2 + 3}{2} + \frac{2 - 3}{2} = 2,$$

- and similarly, in the last round,

$$c = \frac{g_1(4) + g_1(13)}{2} + 5 \cdot \frac{g_1(4) - g_1(13)}{2 \cdot 4} = 14.$$

FRI as a Tree

	1	16	4	13	2	15	8	9
g_0 on Ω_0	11	16	2	0	2	3	13	11

Figure: Every pair $\{x, -x\}$ (neighbouring cells, point written above the value) is folded into the cell of x^2 in the next layer. A query starting at $x_0 = 2$ climbs down one branch (orange): it reads $g_0(\pm 2)$, then $g_1(\pm 4)$, and compares the last fold with c.

FRI as a Tree

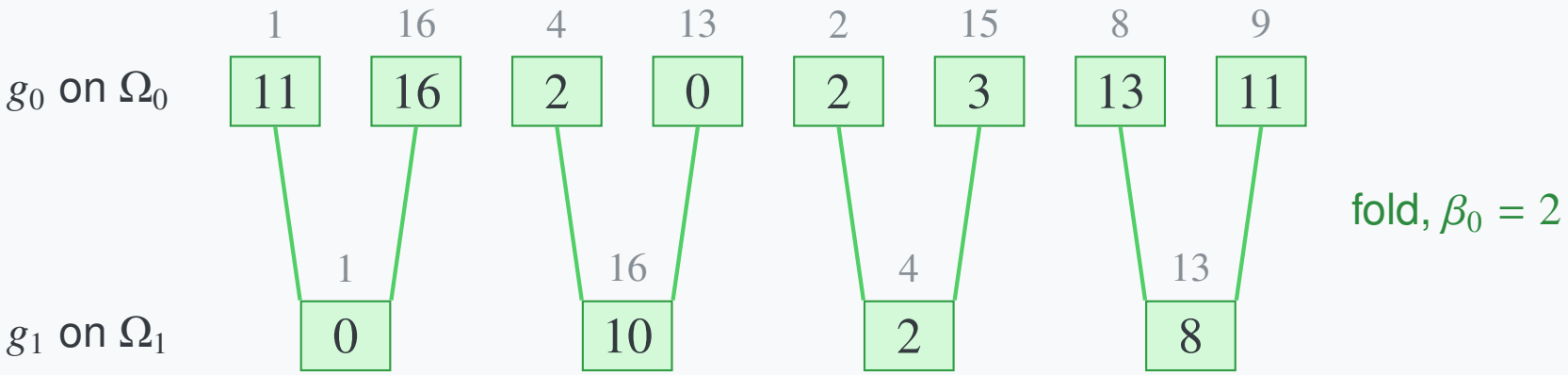


Figure: Every pair $\{x, -x\}$ (neighbouring cells, point written above the value) is folded into the cell of x^2 in the next layer. A query starting at $x_0 = 2$ climbs down one branch (orange): it reads $g_0(\pm 2)$, then $g_1(\pm 4)$, and compares the last fold with c.

FRI as a Tree

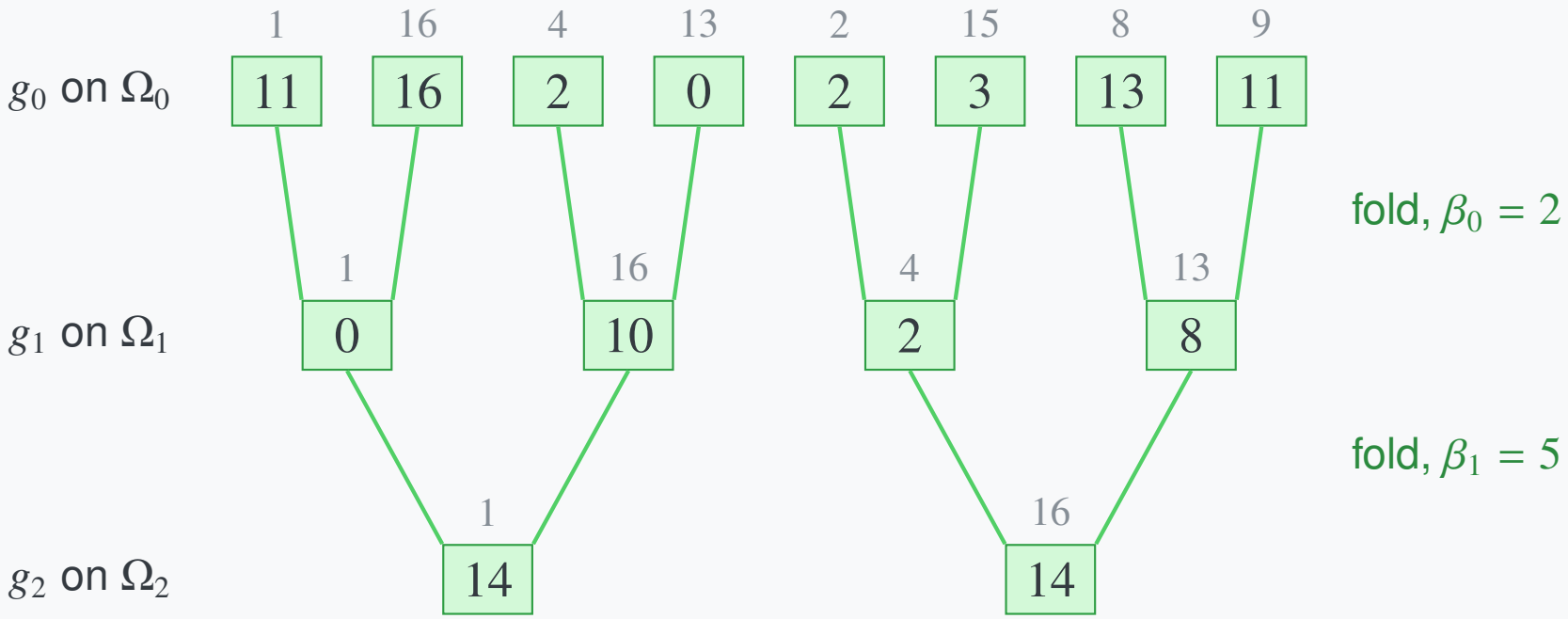


Figure: Every pair $\{x, -x\}$ (neighbouring cells, point written above the value) is folded into the cell of x^2 in the next layer. A query starting at $x_0 = 2$ climbs down one branch (orange): it reads $g_0(\pm 2)$, then $g_1(\pm 4)$, and compares the last fold with c.

FRI as a Tree

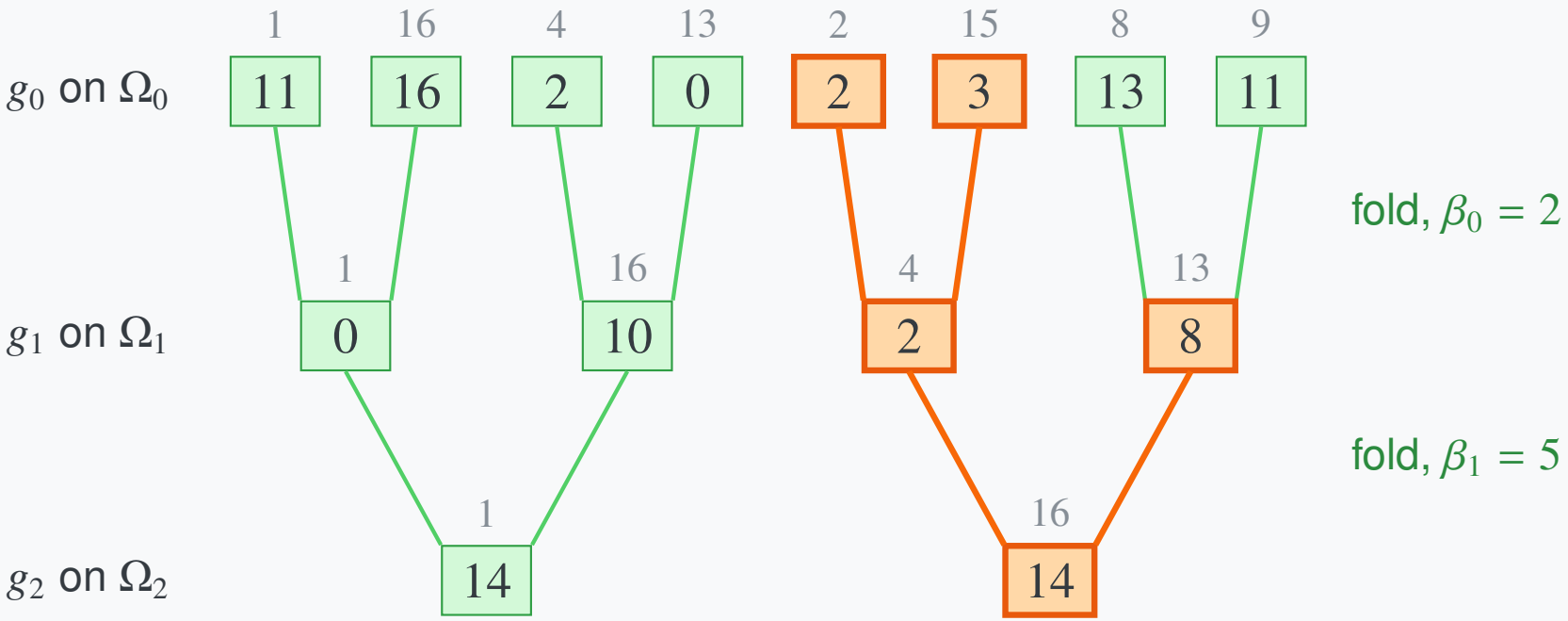


Figure: Every pair $\{x, -x\}$ (neighbouring cells, point written above the value) is folded into the cell of x^2 in the next layer. A query starting at $x_0 = 2$ climbs down one branch (orange): it reads $g_0(\pm 2)$, then $g_1(\pm 4)$, and compares the last fold with c.

A Cheating Prover

Example

The prover tries to pass off $g_0 = \overline{T^5} = (1, 15, 4, 9, 16, 2, 13, 8)$, which is not in RS$[\mathbb{F}_{17}, \Omega, 4]$, folding honestly with $\beta_0 = 2, \beta_1 = 5$.

A Cheating Prover

Example

The prover tries to pass off $g_0 = \overline{T^5} = (1, 15, 4, 9, 16, 2, 13, 8)$, which is not in RS$[\mathbb{F}_{17}, \Omega, 4]$, folding honestly with $\beta_0 = 2, \beta_1 = 5$.

- $T^5 = T \cdot (T^2)^2$, so $g_E = 0$, $g_O = \overline{T^2}$, and $g_1 = \overline{2T^2} = (2, 15, 2, 15)$: the excess degree survived the fold.

A Cheating Prover

Example

The prover tries to pass off $g_0 = \overline{T^5} = (1, 15, 4, 9, 16, 2, 13, 8)$, which is not in RS$[\mathbb{F}_{17}, \Omega, 4]$, folding honestly with $\beta_0 = 2, \beta_1 = 5$.

- $T^5 = T \cdot (T^2)^2$, so $g_E = 0$, $g_O = \overline{T^2}$, and $g_1 = \overline{2T^2} = (2, 15, 2, 15)$: the excess degree survived the fold.
- $g_2 = \overline{2T} = (2, 15)$ on $\Omega_2 = \{1, -1\}$ is not constant: whatever c is sent, each query catches it with probability $1/2$.

A Cheating Prover

Example

The prover tries to pass off $g_0 = \overline{T^5} = (1, 15, 4, 9, 16, 2, 13, 8)$, which is not in RS$[\mathbb{F}_{17}, \Omega, 4]$, folding honestly with $\beta_0 = 2, \beta_1 = 5$.

- $T^5 = T \cdot (T^2)^2$, so $g_E = 0$, $g_O = \overline{T^2}$, and $g_1 = \overline{2T^2} = (2, 15, 2, 15)$: the excess degree survived the fold.
- $g_2 = \overline{2T} = (2, 15)$ on $\Omega_2 = \{1, -1\}$ is not constant: whatever c is sent, each query catches it with probability $1/2$.
- Sending a g_1 that **is** a codeword instead makes g_1 inconsistent with g_0 on many pairs: caught by the first round consistency check.

A Cheating Prover

Example

The prover tries to pass off $g_0 = \overline{T^5} = (1, 15, 4, 9, 16, 2, 13, 8)$, which is not in $\mathrm{RS}[\mathbb{F}_{17}, \Omega, 4]$, folding honestly with $\beta_0 = 2, \beta_1 = 5$.

- $T^5 = T \cdot (T^2)^2$, so $g_E = 0$, $g_O = \overline{T^2}$, and $g_1 = \overline{2T^2} = (2, 15, 2, 15)$: the excess degree survived the fold.
- $g_2 = \overline{2T} = (2, 15)$ on $\Omega_2 = \{1, -1\}$ is not constant: whatever c is sent, each query catches it with probability $1/2$.
- Sending a g_1 that **is** a codeword instead makes g_1 inconsistent with g_0 on many pairs: caught by the first round consistency check.

Question #2

Why must the verifier send β_i only **after** receiving g_i? Describe a cheating strategy if all challenges were known in advance.

Efficiency

- **Prover:** each fold is linear in the size of the current domain, so $O(n + n/2 + \dots) = O(n)$ field operations.

Efficiency

- **Prover:** each fold is linear in the size of the current domain, so $O(n + n/2 + \dots) = O(n)$ field operations.
- **Proof length:** the oracles $g_1, \dots, g_{\ell-1}$ have total length $n/2 + n/4 + \cdots < n$, less than the input.

Efficiency

- **Prover:** each fold is linear in the size of the current domain, so $O(n + n/2 + \dots) = O(n)$ field operations.
- **Proof length:** the oracles $g_1, \dots, g_{\ell-1}$ have total length $n/2 + n/4 + \cdots < n$, less than the input.
- **Queries:** each repetition reads 2 values in each of ℓ rounds, so $2\ell s = O(s \log n)$; the verifier's work is of the same order.

Efficiency

- **Prover:** each fold is linear in the size of the current domain, so $O(n + n/2 + \dots) = O(n)$ field operations.
- **Proof length:** the oracles $g_1, \dots, g_{\ell-1}$ have total length $n/2 + n/4 + \cdots < n$, less than the input.
- **Queries:** each repetition reads 2 values in each of ℓ rounds, so $2\ell s = O(s \log n)$; the verifier's work is of the same order.

Question #3

Every query reads the pair $g_i(x_i), g_i(-x_i)$, and each value comes with a Merkle authentication path. How should the prover arrange the leaves of the Merkle tree of g_i to halve the number of paths?

Soundness: Intuition

Suppose g_0 is far from the code. In every round of the commit phase:

- if the prover folds honestly, then by the folding lemma the fold stays far, except with probability ε_δ over β_i;

Soundness: Intuition

Suppose g_0 is far from the code. In every round of the commit phase:

- if the prover folds honestly, then by the folding lemma the fold stays far, except with probability ε_δ over β_i;
- if the prover sends a g_{i+1} that is close to the code, then g_{i+1} differs from the true fold of g_i on many points, and round consistency detects this at a random point.

Soundness: Intuition

Suppose g_0 is far from the code. In every round of the commit phase:

- if the prover folds honestly, then by the folding lemma the fold stays far, except with probability ε_δ over β_i;
- if the prover sends a g_{i+1} that is close to the code, then g_{i+1} differs from the true fold of g_i on many points, and round consistency detects this at a random point.

Either way

A far input leaves a trace of inconsistencies of density about δ along the path of a random query.

Soundness

Theorem (FRI soundness, informal)

Let g_0 be δ-far from RS$[\mathbb{F}, \Omega, k]$. For every prover, the FRI verifier with s repetitions accepts with probability at most

$$\ell \cdot \varepsilon_\delta + (1 - \delta)^s,$$

for every δ below the Johnson bound $1 - \sqrt{\rho}$.

Soundness

Theorem (FRI soundness, informal)

Let g_0 be δ-far from RS$[\mathbb{F}, \Omega, k]$. For every prover, the FRI verifier with s repetitions accepts with probability at most

$$\ell \cdot \varepsilon_\delta + (1 - \delta)^s,$$

for every δ below the Johnson bound $1 - \sqrt{\rho}$.

- $\ell \cdot \varepsilon_\delta$: some β_i is unlucky — negligible for a large field.
- $(1 - \delta)^s$: all s query paths miss — this determines the number of queries.

Soundness: History

Remark

The original analysis gives a weaker bound: $3n/\#\mathbb{F} + (1 - \min\{\delta, \delta_0\})^s$, where $\delta_0 \approx (1 - 3\rho)/4$ lies below the unique decoding radius.

Soundness: History

Remark

The original analysis gives a weaker bound: $3n/\#\mathbb{F} + (1 - \min\{\delta, \delta_0\})^s$, where $\delta_0 \approx (1 - 3\rho)/4$ lies below the unique decoding radius.

- The improvement to the Johnson bound came with DEEP-FRI, and then with the proximity gap theorem (BCIKS20).

Soundness: History

Remark

The original analysis gives a weaker bound: $3n/\#\mathbb{F} + (1 - \min\{\delta, \delta_0\})^s$, where $\delta_0 \approx (1 - 3\rho)/4$ lies below the unique decoding radius.

- The improvement to the Johnson bound came with DEEP-FRI, and then with the proximity gap theorem (BCIKS20).
- The original FRI paper already conjectured that the bound holds up to the capacity $1 - \rho$.

Soundness: History

Remark

The original analysis gives a weaker bound: $3n/\#\mathbb{F} + (1 - \min\{\delta, \delta_0\})^s$, where $\delta_0 \approx (1 - 3\rho)/4$ lies below the unique decoding radius.

- The improvement to the Johnson bound came with DEEP-FRI, and then with the proximity gap theorem (BCIKS20).
- The original FRI paper already conjectured that the bound holds up to the capacity $1 - \rho$.

Question #4

Let $\rho = 1/4$ and ignore $\ell \cdot \varepsilon_\delta$. Find the smallest s with remaining term at most 2^{-100}: (a) original bound, (b) δ approaching the Johnson bound, (c) δ approaching the capacity $1 - \rho$.

Soundness: History

Remark

The original analysis gives a weaker bound: $3n/\#\mathbb{F} + (1 - \min\{\delta, \delta_0\})^s$, where $\delta_0 \approx (1 - 3\rho)/4$ lies below the unique decoding radius.

- The improvement to the Johnson bound came with DEEP-FRI, and then with the proximity gap theorem (BCIKS20).
- The original FRI paper already conjectured that the bound holds up to the capacity $1 - \rho$.

Question #4

Let $\rho = 1/4$ and ignore $\ell \cdot \varepsilon_\delta$. Find the smallest s with remaining term at most 2^{-100}: (a) original bound, (b) δ approaching the Johnson bound, (c) δ approaching the capacity $1 - \rho$.

// Answer: $s \geq 100/\log_2(1/(1-\delta))$, *about* 1074, 100 *and* 50 *repetitions.*

FRI in the Big Picture

- The input of the RS-IOPP is the **virtual** $h = \sum_j \alpha^j \mathrm{q}_j$. Whenever the verifier needs $h(\pm x_0)$, it queries every committed oracle at $\pm x_0$ and computes $h(\pm x_0)$ itself.

FRI in the Big Picture

- The input of the RS-IOPP is the **virtual** $h = \sum_j \alpha^j \mathsf{q}_j$. Whenever the verifier needs $h(\pm x_0)$, it queries every committed oracle at $\pm x_0$ and computes $h(\pm x_0)$ itself.
- The first layer of FRI comes for free: only $g_1, \ldots, g_{\ell-1}$ and c are new prover messages.

FRI in the Big Picture

- The input of the RS-IOPP is the **virtual** $h = \sum_j \alpha^j \mathsf{q}_j$. Whenever the verifier needs $h(\pm x_0)$, it queries every committed oracle at $\pm x_0$ and computes $h(\pm x_0)$ itself.
- The first layer of FRI comes for free: only $g_1, \ldots, g_{\ell-1}$ and c are new prover messages.
- **BCS transformation** (Ben-Sasson, Chiesa, Spooner) turns the IOP into a SNARK:
  1: every oracle is replaced by a Merkle root, every query is answered with an authentication path;
  2: every challenge, including the query points x_0, is derived by hashing the transcript (Fiat–Shamir).

FRI in the Big Picture

- The input of the RS-IOPP is the **virtual** $h = \sum_j \alpha^j \mathrm{q}_j$. Whenever the verifier needs $h(\pm x_0)$, it queries every committed oracle at $\pm x_0$ and computes $h(\pm x_0)$ itself.
- The first layer of FRI comes for free: only $g_1, \ldots, g_{\ell-1}$ and c are new prover messages.
- **BCS transformation** (Ben-Sasson, Chiesa, Spooner) turns the IOP into a SNARK:
  1: every oracle is replaced by a Merkle root, every query is answered with an authentication path;
  2: every challenge, including the query points x_0, is derived by hashing the transcript (Fiat–Shamir).



Folding Factor

- Fold whole cosets of size 2^η at once, using $x \mapsto x^{2^\eta}$: the degree drops by 2^η in one round.

Folding Factor

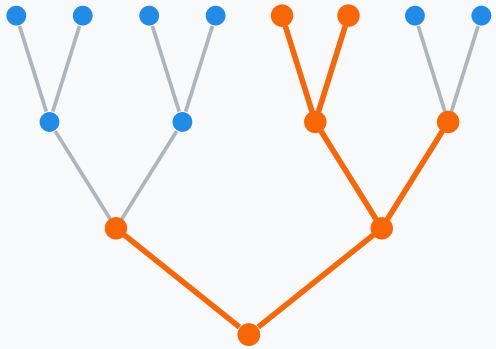
- Fold whole cosets of size 2^η at once, using $x \mapsto x^{2^\eta}$: the degree drops by 2^η in one round.
- The fold is the value at β_i of the polynomial of degree $< 2^\eta$ interpolating g_i on the coset ($\eta = 1$ is the line L).

Folding Factor

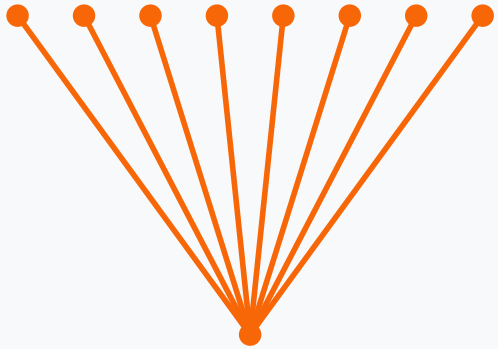
- Fold whole cosets of size 2^η at once, using $x \mapsto x^{2^\eta}$: the degree drops by 2^η in one round.
- The fold is the value at β_i of the polynomial of degree $< 2^\eta$ interpolating g_i on the coset ($\eta = 1$ is the line L).
- A query reads slightly more values, but they all sit in **one Merkle leaf**. Typically $\eta \in \{2, 3, 4\}$.

Folding Factor

- Fold whole cosets of size 2^η at once, using $x \mapsto x^{2^\eta}$: the degree drops by 2^η in one round.
- The fold is the value at β_i of the polynomial of degree $< 2^\eta$ interpolating g_i on the coset ($\eta = 1$ is the line L).
- A query reads slightly more values, but they all sit in **one Merkle leaf**. Typically $\eta \in \{2, 3, 4\}$.



2-to-1 folding, 3 rounds:
6 values in 3 Merkle openings



8-to-1 folding, 1 round:
8 values in 1 Merkle opening

Other Variants

- **Stop early:** stop folding before reaching a constant and send the remaining polynomial of small degree in full (as the original paper does).

Other Variants

- **Stop early:** stop folding before reaching a constant and send the remaining polynomial of small degree in full (as the original paper does).
- **Other domains:** the original FRI is stated for binary fields, with additive cosets and a subspace-vanishing polynomial instead of $x \mapsto x^2$.

Other Variants

- **Stop early:** stop folding before reaching a constant and send the remaining polynomial of small degree in full (as the original paper does).
- **Other domains:** the original FRI is stated for binary fields, with additive cosets and a subspace-vanishing polynomial instead of $x \mapsto x^2$.
- **Circle STARKs:** over $p = 2^{31} - 1$, where $p - 1$ has no large power of two; use the points of the circle $x^2 + y^2 = 1$, a group of order $p + 1 = 2^{31}$.

Other Variants

- **Stop early:** stop folding before reaching a constant and send the remaining polynomial of small degree in full (as the original paper does).
- **Other domains:** the original FRI is stated for binary fields, with additive cosets and a subspace-vanishing polynomial instead of $x \mapsto x^2$.
- **Circle STARKs:** over $p = 2^{31} - 1$, where $p - 1$ has no large power of two; use the points of the circle $x^2 + y^2 = 1$, a group of order $p + 1 = 2^{31}$.
- **STIR:** lowers the rate in every round in addition to the degree, using out-of-domain sampling and quotienting inside every round.

Other Variants

- **Stop early:** stop folding before reaching a constant and send the remaining polynomial of small degree in full (as the original paper does).
- **Other domains:** the original FRI is stated for binary fields, with additive cosets and a subspace-vanishing polynomial instead of $x \mapsto x^2$.
- **Circle STARKs:** over $p = 2^{31} - 1$, where $p - 1$ has no large power of two; use the points of the circle $x^2 + y^2 = 1$, a group of order $p + 1 = 2^{31}$.
- **STIR:** lowers the rate in every round in addition to the degree, using out-of-domain sampling and quotienting inside every round.
- **WHIR** and **Basefold:** the same ideas for multilinear polynomials, combined with sum-check-based Poly-IOPs.

Arithmetic Intermediate Representation

What is Still Missing?

- We have a hash-based compiler: Poly-IOP $\rightarrow$ IOP with one FRI run $\rightarrow$ SNARK.

What is Still Missing?

- We have a hash-based compiler: Poly-IOP $\rightarrow$ IOP with one FRI run $\rightarrow$ SNARK.
- Missing: the Poly-IOP itself, expressing “this computation was performed correctly” as claims about polynomials.

What is Still Missing?

- We have a hash-based compiler: Poly-IOP $\rightarrow$ IOP with one FRI run $\rightarrow$ SNARK.
- Missing: the Poly-IOP itself, expressing "this computation was performed correctly" as claims about polynomials.
- PlonK did this for arithmetic circuits. STARKs target computations that **repeat the same step** many times: a virtual machine, rounds of a hash function.

What is Still Missing?

- We have a hash-based compiler: Poly-IOP → IOP with one FRI run → SNARK.
- Missing: the Poly-IOP itself, expressing "this computation was performed correctly" as claims about polynomials.
- PlonK did this for arithmetic circuits. STARKs target computations that **repeat the same step** many times: a virtual machine, rounds of a hash function.

Definition

The **arithmetic intermediate representation (AIR)** describes a computation in terms of an execution trace that satisfies constraints induced by the correct evolution of the state.

What is Still Missing?

- We have a hash-based compiler: Poly-IOP → IOP with one FRI run → SNARK.
- Missing: the Poly-IOP itself, expressing "this computation was performed correctly" as claims about polynomials.
- PlonK did this for arithmetic circuits. STARKs target computations that **repeat the same step** many times: a virtual machine, rounds of a hash function.

Definition

The **arithmetic intermediate representation (AIR)** describes a computation in terms of an execution trace that satisfies constraints induced by the correct evolution of the state.

Pipeline: AIR → Poly-IOP → IOP → SNARK.

Execution Trace

Definition

Suppose the computation evolves a state of w registers for τ cycles. The **algebraic execution trace (AET)** is the table $W \in \mathbb{F}^{\tau \times w}$: rows are states at given times, columns track registers.

Execution Trace

Definition

Suppose the computation evolves a state of w registers for τ cycles. The **algebraic execution trace (AET)** is the table $W \in \mathbb{F}^{\tau \times w}$: rows are states at given times, columns track registers.

Definition

A **state transition function** $\phi : \mathbb{F}^w \to \mathbb{F}^w$ determines the next state from the previous one. A set of **boundary conditions**

$$\mathcal{B} \subseteq [\tau] \times [w] \times \mathbb{F}$$

fixes some registers at some cycles: $(i, j, v) \in \mathcal{B}$ means that register j holds v at cycle i.

Running Example: Fibonacci Squares

Example

Over $\mathbb{F}_{17}$: $a_{i+2} = a_{i+1}^2 + a_i^2$ with $a_0 = 1$ and $a_1 = s$. The prover claims to know s such that $a_4 = 13$.

Running Example: Fibonacci Squares

Example

Over $\mathbb{F}_{17}$: $a_{i+2} = a_{i+1}^2 + a_i^2$ with $a_0 = 1$ and $a_1 = s$. The prover claims to know s such that $a_4 = 13$.

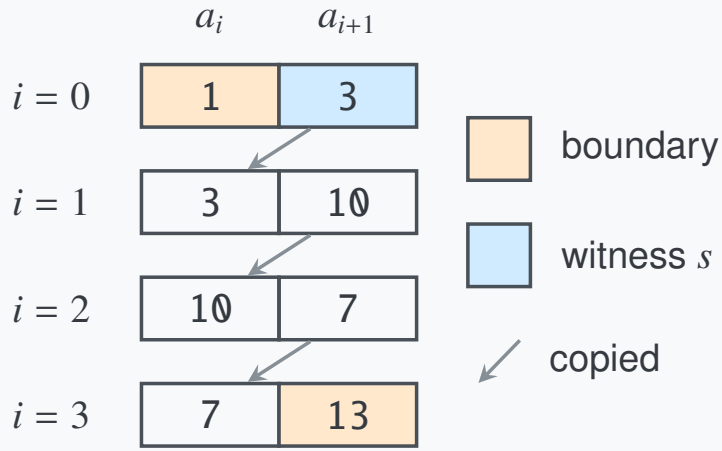
Indeed, $s = 3$ works: $a_2 = 1^2 + 3^2 = 10$, $a_3 = 3^2 + 10^2 = 7$, $a_4 = 10^2 + 7^2 = 13$.

Running Example: Fibonacci Squares

Example

Over $\mathbb{F}_{17}$: $a_{i+2} = a_{i+1}^2 + a_i^2$ with $a_0 = 1$ and $a_1 = s$. The prover claims to know s such that $a_4 = 13$.

Indeed, $s = 3$ works: $a_2 = 1^2 + 3^2 = 10$, $a_3 = 3^2 + 10^2 = 7$, $a_4 = 10^2 + 7^2 = 13$.



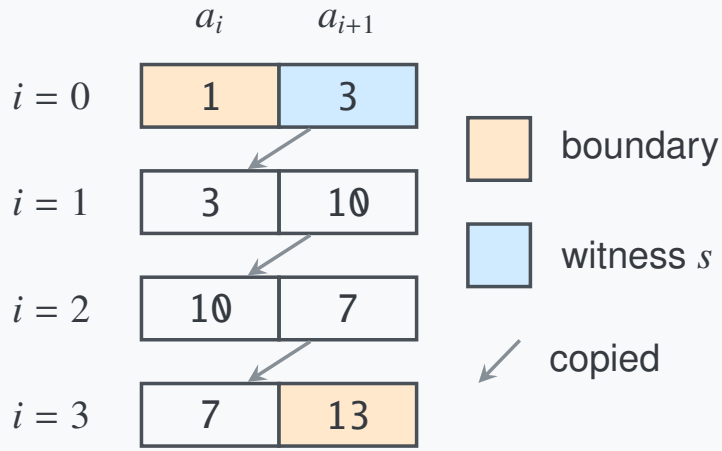
$w = 2$ registers, $\tau = 4$ cycles:
$W_{[i,0]} = a_i$, $W_{[i,1]} = a_{i+1}$.

Running Example: Fibonacci Squares

Example

Over $\mathbb{F}_{17}$: $a_{i+2} = a_{i+1}^2 + a_i^2$ with $a_0 = 1$ and $a_1 = s$. The prover claims to know s such that $a_4 = 13$.

Indeed, $s = 3$ works: $a_2 = 1^2 + 3^2 = 10$, $a_3 = 3^2 + 10^2 = 7$, $a_4 = 10^2 + 7^2 = 13$.



$w = 2$ registers, $\tau = 4$ cycles:
$W_{[i,0]} = a_i$, $W_{[i,1]} = a_{i+1}$.
$\phi(x_0, x_1) = (x_1, x_0^2 + x_1^2)$,
$\mathcal{B} = \{(0, 0, 1), (3, 1, 13)\}$.

Computational Integrity Claim

The claim consists of ϕ and $\mathcal{B}$; the witness is W. The claim is **true** if there is $W \in \mathbb{F}^{\tau \times w}$ such that:

- $\phi(W_{[i,:]}) = W_{[i+1,:]}$ for all $i \in [\tau - 1]$;
- $W_{[i,j]} = v$ for all $(i, j, v) \in \mathcal{B}$.

Computational Integrity Claim

The claim consists of ϕ and $\mathcal{B}$; the witness is W. The claim is **true** if there is $W \in \mathbb{F}^{\tau \times w}$ such that:

- $\phi(W_{[i,:]}) = W_{[i+1,:]}$ for all $i \in [\tau - 1]$;
- $W_{[i,j]} = v$ for all $(i, j, v) \in \mathcal{B}$.

ϕ must be described by low-degree polynomials independent of the cycle, which only need to **distinguish** correct evolutions from incorrect ones:

Computational Integrity Claim

The claim consists of ϕ and $\mathcal{B}$; the witness is W. The claim is **true** if there is $W \in \mathbb{F}^{\tau \times w}$ such that:

- $\phi(W_{[i,:]}) = W_{[i+1,:]}$ for all $i \in [\tau - 1]$;
- $W_{[i,j]} = v$ for all $(i, j, v) \in \mathcal{B}$.

ϕ must be described by low-degree polynomials independent of the cycle, which only need to **distinguish** correct evolutions from incorrect ones:

Definition

Transition verification polynomials $p_u(X_0, \ldots, X_{w-1}, Y_0, \ldots, Y_{w-1})$, $u \in [\kappa]$, satisfy $\phi(\mathbf{x}) = \mathbf{y}$ iff $p_u(\mathbf{x}, \mathbf{y}) = 0$ for all $u \in [\kappa]$.

Computational Integrity Claim

The claim consists of ϕ and $\mathcal{B}$; the witness is W. The claim is **true** if there is $W \in \mathbb{F}^{\tau \times w}$ such that:

- $\phi(W_{[i,:]}) = W_{[i+1,:]}$ for all $i \in [\tau - 1]$;
- $W_{[i,j]} = v$ for all $(i, j, v) \in \mathcal{B}$.

ϕ must be described by low-degree polynomials independent of the cycle, which only need to **distinguish** correct evolutions from incorrect ones:

Definition

Transition verification polynomials $p_u(X_0, \ldots, X_{w-1}, Y_0, \ldots, Y_{w-1})$, $u \in [\kappa]$, satisfy $\phi(\mathbf{x}) = \mathbf{y}$ iff $p_u(\mathbf{x}, \mathbf{y}) = 0$ for all $u \in [\kappa]$.

Transition constraints: for all $i \in [\tau - 1]$ and $u \in [\kappa]$,

$$p_u(W_{[i,0]}, \ldots, W_{[i,w-1]}, W_{[i+1,0]}, \ldots, W_{[i+1,w-1]}) = 0.$$

Transition Constraints: Example

Example

The transition is verified by $\kappa = 2$ polynomials

$$p_0 = Y_0 - X_1, \qquad p_1 = Y_1 - X_0^2 - X_1^2 :$$

a_{i+1} is copied to the next row, and the recurrence itself. At cycle $i = 0$:

$$p_0(W_{[0,0]}, W_{[0,1]}, W_{[1,0]}, W_{[1,1]}) = 3 - 3 = 0,$$
$$p_1(W_{[0,0]}, W_{[0,1]}, W_{[1,0]}, W_{[1,1]}) = 10 - 1^2 - 3^2 = 0.$$

Transition Constraints: Example

Example

The transition is verified by $\kappa = 2$ polynomials

$$p_0 = Y_0 - X_1, \qquad p_1 = Y_1 - X_0^2 - X_1^2 :$$

a_{i+1} is copied to the next row, and the recurrence itself. At cycle $i = 0$:

$$p_0(W_{[0,0]}, W_{[0,1]}, W_{[1,0]}, W_{[1,1]}) = 3 - 3 = 0,$$
$$p_1(W_{[0,0]}, W_{[0,1]}, W_{[1,0]}, W_{[1,1]}) = 10 - 1^2 - 3^2 = 0.$$

Non-determinism: $y = x^{-1}$ requires computing $x^{\#\mathbb{F}-2}$, but it is verified by the degree-2 polynomial $XY - 1$.

AIR vs Arithmetic Circuits

- A circuit can describe an arbitrary computation, but every gate needs its own constraint.

AIR vs Arithmetic Circuits

- A circuit can describe an arbitrary computation, but every gate needs its own constraint.
- An AIR describes only computations that repeat one step, but the same κ polynomials describe all $\tau - 1$ steps: the description of the computation does not grow with τ.

AIR vs Arithmetic Circuits

- A circuit can describe an arbitrary computation, but every gate needs its own constraint.
- An AIR describes only computations that repeat one step, but the same κ polynomials describe all $\tau - 1$ steps: the description of the computation does not grow with τ.

Remark

This uniformity matches the cyclic structure of the evaluation domains perfectly: moving to the next row will become multiplication of the argument by a root of unity.

Trace Interpolation

To get a Poly-IOP, represent the **witness** by polynomials as well.

Trace Interpolation

To get a Poly-IOP, represent the **witness** by polynomials as well.

- Let ω_τ be a primitive τth root of unity and $H := \langle \omega_\tau \rangle$ the **trace domain** (τ a power of two, otherwise pad).

Trace Interpolation

To get a Poly-IOP, represent the **witness** by polynomials as well.

- Let ω_τ be a primitive τth root of unity and $H := \langle \omega_\tau \rangle$ the **trace domain** (τ a power of two, otherwise pad).
- The **trace polynomial** $t_j \in \mathbb{F}[T]^{<\tau}$ of register $j \in [w]$ is the unique polynomial with

$$t_j(\omega_\tau^i) = W_{[i,j]} \quad \text{for all } i \in [\tau],$$

computed with the inverse NTT: the jth column of W is the restriction of t_j to H.

Trace Interpolation

To get a Poly-IOP, represent the **witness** by polynomials as well.

- Let ω_τ be a primitive τth root of unity and $H := \langle \omega_\tau \rangle$ the **trace domain** (τ a power of two, otherwise pad).
- The **trace polynomial** $t_j \in \mathbb{F}[T]^{<\tau}$ of register $j \in [w]$ is the unique polynomial with

$$t_j(\omega_\tau^i) = W_{[i,j]} \quad \text{for all } i \in [\tau],$$

computed with the inverse NTT: the jth column of W is the restriction of t_j to H.

Example

$H = \langle 13 \rangle = \{1, 13, 16, 4\}$, $\tau = 4$, $\omega_\tau = 13$. Interpolating the columns:

$$t_0(T) = 1 + 15T + 13T^2 + 6T^3, \qquad t_1(T) = 4 + 13T + T^2 + 2T^3.$$

For instance, $t_0(\omega_\tau^2) = t_0(-1) = 1 - 15 + 13 - 6 = 10 = W_{[2,0]}$.

Constraint Polynomials

On the trace polynomials, the claim becomes:

- boundary: $t_j(\omega_\tau^i) = v$ for all $(i, j, v) \in \mathcal{B}$;
- transition: for all $i \in [\tau - 1]$ and $u \in [\kappa]$,

$$p_u(t_0(\omega_\tau^i), \ldots, t_{w-1}(\omega_\tau^i), t_0(\omega_\tau^{i+1}), \ldots, t_{w-1}(\omega_\tau^{i+1})) = 0.$$

Constraint Polynomials

On the trace polynomials, the claim becomes:

- boundary: $t_j(\omega_\tau^i) = v$ for all $(i, j, v) \in \mathcal{B}$;
- transition: for all $i \in [\tau - 1]$ and $u \in [\kappa]$,

$$p_u(t_0(\omega_\tau^i), \ldots, t_{w-1}(\omega_\tau^i), t_0(\omega_\tau^{i+1}), \ldots, t_{w-1}(\omega_\tau^{i+1})) = 0.$$

The structure of H pays off

The next row is reached by multiplying the argument by ω_τ: $\omega_\tau^{i+1} = \omega_\tau \cdot \omega_\tau^i$.

Constraint Polynomials

On the trace polynomials, the claim becomes:

- boundary: $t_j(\omega_\tau^i) = v$ for all $(i, j, v) \in \mathcal{B}$;
- transition: for all $i \in [\tau - 1]$ and $u \in [\kappa]$,

$$p_u(t_0(\omega_\tau^i), \ldots, t_{w-1}(\omega_\tau^i), t_0(\omega_\tau^{i+1}), \ldots, t_{w-1}(\omega_\tau^{i+1})) = 0.$$

The structure of H pays off

The next row is reached by multiplying the argument by ω_τ: $\omega_\tau^{i+1} = \omega_\tau \cdot \omega_\tau^i$.

So the transition constraint is the **constraint polynomial**

$$c_u(T) := p_u(t_0(T), \ldots, t_{w-1}(T), t_0(\omega_\tau T), \ldots, t_{w-1}(\omega_\tau T))$$

vanishing on $\{\omega_\tau^i : i \in [\tau - 1]\}$: all of H except the last point.

Generalized AIR Constraint

A polynomial vanishes on a set S iff it is divisible by $\prod_{x \in S}(T - x)$: exactly the idea behind quotienting.

Generalized AIR Constraint

A polynomial vanishes on a set S iff it is divisible by $\prod_{x \in S}(T - x)$: exactly the idea behind quotienting.

Definition (Generalized AIR constraint)

A **generalized AIR constraint** is given by two polynomials:

- the **numerator**: which equation between trace elements holds, independently of the cycle (transition: c_u; boundary of register j: $t_j - \iota_j$, where ι_j interpolates the boundary values);
- the **denominator**: a vanishing polynomial of the points where it must hold (transition: H without the last point; boundary: the boundary points).

The constraint is satisfied iff the numerator is divisible by the denominator, that is, the **quotient** is a polynomial.

Boundary and Transition Quotients

For every register $j \in [w]$ (the ι and ν of multiple quotienting, for the points $\mathcal{B}_j$):

- $\mathcal{B}_j := \{\omega_\tau^i : (i,j,v) \in \mathcal{B}\}$, the boundary points;
- the **boundary interpolant** ι_j, $\deg \iota_j < \#\mathcal{B}_j$, with $\iota_j(\omega_\tau^i) = v$ for all $(i,j,v) \in \mathcal{B}$;
- $\nu_j(T) := \prod_{x \in \mathcal{B}_j}(T - x)$.

Boundary and Transition Quotients

For every register $j \in [w]$ (the ι and ν of multiple quotienting, for the points $\mathcal{B}_j$):

- $\mathcal{B}_j := \{\omega_\tau^i : (i,j,v) \in \mathcal{B}\}$, the boundary points;
- the **boundary interpolant** ι_j, $\deg \iota_j < \#\mathcal{B}_j$, with $\iota_j(\omega_\tau^i) = v$ for all $(i,j,v) \in \mathcal{B}$;
- $\nu_j(T) := \prod_{x \in \mathcal{B}_j}(T - x)$.

The **transition vanishing polynomial** is

$$\nu_{\mathcal{T}}(T) := \prod_{i \in [\tau - 1]} (T - \omega_\tau^i) = \frac{T^\tau - 1}{T - \omega_\tau^{\tau-1}}.$$

Boundary and Transition Quotients

For every register $j \in [w]$ (the ι and ν of multiple quotienting, for the points $\mathcal{B}_j$):

- $\mathcal{B}_j := \{\omega_\tau^i : (i,j,v) \in \mathcal{B}\}$, the boundary points;
- the **boundary interpolant** ι_j, $\deg \iota_j < \#\mathcal{B}_j$, with $\iota_j(\omega_\tau^i) = v$ for all $(i,j,v) \in \mathcal{B}$;
- $\nu_j(T) := \prod_{x \in \mathcal{B}_j}(T - x)$.

The **transition vanishing polynomial** is

$$\nu_{\mathcal{T}}(T) := \prod_{i \in [\tau-1]} (T - \omega_\tau^i) = \frac{T^\tau - 1}{T - \omega_\tau^{\tau-1}}.$$

The **quotients** are

$$q_j^{\mathcal{B}} := \frac{t_j - \iota_j}{\nu_j}, \quad j \in [w], \qquad q_u^{\mathcal{T}} := \frac{c_u}{\nu_{\mathcal{T}}}, \quad u \in [\kappa].$$

Constraints as Divisibility

Proposition

The trace W satisfies all boundary and transition constraints if and only if all boundary quotients $q_j^{\mathcal{B}}$ and all transition quotients $q_u^{\mathcal{T}}$ are polynomials.

Constraints as Divisibility

Proposition

The trace W satisfies all boundary and transition constraints if and only if all boundary quotients $q_j^{\mathcal{B}}$ and all transition quotients $q_u^{\mathcal{T}}$ are polynomials.

Proof. A quotient is a polynomial iff its numerator vanishes at every root of its denominator:

- boundary: $t_j(\omega_\tau^i) = v$ for all $(i, j, v) \in \mathcal{B}$;
- transition: $c_u(\omega_\tau^i) = 0$ for all $i \in [\tau - 1]$,

which are exactly the constraints. □

Constraints as Divisibility

Proposition

The trace W satisfies all boundary and transition constraints if and only if all boundary quotients $q_j^{\mathcal{B}}$ and all transition quotients $q_u^{\mathcal{T}}$ are polynomials.

Proof. A quotient is a polynomial iff its numerator vanishes at every root of its denominator:

- boundary: $t_j(\omega_\tau^i) = v$ for all $(i, j, v) \in \mathcal{B}$;
- transition: $c_u(\omega_\tau^i) = 0$ for all $i \in [\tau - 1]$,

which are exactly the constraints. □

Degrees

If p_u has total degree d_u, then $\deg c_u \leq d_u(\tau - 1)$, so a valid trace gives $\deg q_u^{\mathcal{T}} \leq (d_u - 1)(\tau - 1)$ and $\deg q_j^{\mathcal{B}} < \tau - \#\mathcal{B}_j$.

Constraints as Divisibility: Example

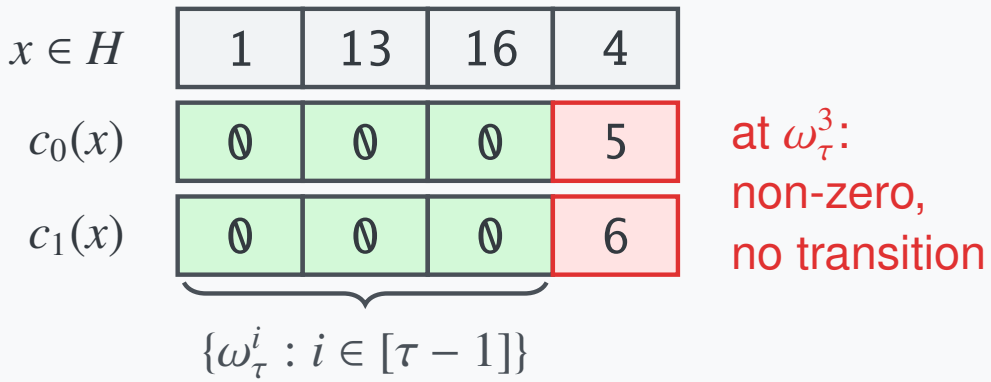
Substituting $X_j \mapsto t_j(T)$ and $Y_j \mapsto t_j(\omega_\tau T)$:

$$
\begin{aligned}
c_0(T) &= t_0(\omega_\tau T) - t_1(T) = 14 + 12T + 3T^2 + 5T^3, \\
c_1(T) &= t_1(\omega_\tau T) - t_0(T)^2 - t_1(T)^2 \\
&= 4 + T + 13T^2 + 6T^3 + 6T^4 + 10T^5 + 11T^6.
\end{aligned}
$$

Constraints as Divisibility: Example

Substituting $X_j \mapsto t_j(T)$ and $Y_j \mapsto t_j(\omega_\tau T)$:

$$c_0(T) = t_0(\omega_\tau T) - t_1(T) = 14 + 12T + 3T^2 + 5T^3,$$
$$c_1(T) = t_1(\omega_\tau T) - t_0(T)^2 - t_1(T)^2$$
$$= 4 + T + 13T^2 + 6T^3 + 6T^4 + 10T^5 + 11T^6.$$



Both vanish on $\omega_\tau^0, \omega_\tau^1, \omega_\tau^2$, but not on ω_τ^3: there is no next row to compare with.

Constraints as Divisibility: Example

- Boundary conditions $(0, 0, 1)$ and $(3, 1, 13)$; $\omega_\tau^0 = 1$, $\omega_\tau^3 = 4$, so $\mathcal{B}_0 = \{1\}$, $\mathcal{B}_1 = \{4\}$, and

$$\iota_0 = 1, \quad \iota_1 = 13, \quad \nu_0 = T - 1, \quad \nu_1 = T - 4.$$

Constraints as Divisibility: Example

- Boundary conditions $(0, 0, 1)$ and $(3, 1, 13)$; $\omega_\tau^0 = 1$, $\omega_\tau^3 = 4$, so $\mathcal{B}_0 = \{1\}$, $\mathcal{B}_1 = \{4\}$, and

$$\iota_0 = 1, \quad \iota_1 = 13, \quad \nu_0 = T - 1, \quad \nu_1 = T - 4.$$

- The transition vanishing polynomial:

$$\nu_{\mathcal{T}}(T) = (T-1)(T-13)(T-16) = \frac{T^4 - 1}{T - 4} = T^3 + 4T^2 + 16T + 13.$$

Constraints as Divisibility: Example

- Boundary conditions $(0, 0, 1)$ and $(3, 1, 13)$; $\omega_\tau^0 = 1$, $\omega_\tau^3 = 4$, so $\mathcal{B}_0 = \{1\}$, $\mathcal{B}_1 = \{4\}$, and

$$\iota_0 = 1, \quad \iota_1 = 13, \quad \nu_0 = T - 1, \quad \nu_1 = T - 4.$$

- The transition vanishing polynomial:

$$\nu_{\mathcal{T}}(T) = (T-1)(T-13)(T-16) = \frac{T^4 - 1}{T - 4} = T^3 + 4T^2 + 16T + 13.$$

All four divisions are exact:

$$q_0^{\mathcal{B}} = \frac{t_0 - 1}{T - 1} = 2T + 6T^2, \qquad q_1^{\mathcal{B}} = \frac{t_1 - 13}{T - 4} = 15 + 9T + 2T^2,$$

$$q_0^{\mathcal{T}} = \frac{c_0}{\nu_{\mathcal{T}}} = 5, \qquad q_1^{\mathcal{T}} = \frac{c_1}{\nu_{\mathcal{T}}} = 16 + 11T^3.$$

AIR as a Poly-IOP

Batch all quotients with a random challenge and check the identity at a random point. Assume $\deg p_u \leq 2$: all quotients have degree $< \tau$.

AIR as a Poly-IOP

Batch all quotients with a random challenge and check the identity at a random point. Assume $\deg p_u \leq 2$: all quotients have degree $< \tau$.

Definition (AIR Poly-IOP)

The AIR $(\tau, w, \{p_u\}_{u\in[\kappa]}, \mathcal{B})$ is public; the prover knows W. We write $\mathbf{t}(x) := (t_0(x), \ldots, t_{w-1}(x))$.

1: $\mathcal{P}$ sends the trace polynomials $t_0, \ldots, t_{w-1} \in \mathbb{F}[T]^{<\tau}$.

AIR as a Poly-IOP

Batch all quotients with a random challenge and check the identity at a random point. Assume $\deg p_u \leq 2$: all quotients have degree $< \tau$.

Definition (AIR Poly-IOP)

The AIR $(\tau, w, \{p_u\}_{u\in[\kappa]}, \mathcal{B})$ is public; the prover knows W. We write $\mathbf{t}(x) := (t_0(x), \ldots, t_{w-1}(x))$.

1: $\mathcal{P}$ sends the trace polynomials $t_0, \ldots, t_{w-1} \in \mathbb{F}[T]^{<\tau}$.

2: $\mathcal{V}$ samples $\gamma \leftarrow\!\$\ \mathbb{F}$; $\mathcal{P}$ sends the **composition polynomial**

$$\psi := \sum_{u\in[\kappa]} \gamma^u \cdot q_u^{\mathcal{T}} + \sum_{j\in[w]} \gamma^{\kappa+j} \cdot q_j^{\mathcal{B}} \in \mathbb{F}[T]^{<\tau}.$$

AIR as a Poly-IOP

Batch all quotients with a random challenge and check the identity at a random point. Assume $\deg p_u \leq 2$: all quotients have degree $< \tau$.

Definition (AIR Poly-IOP)

The AIR $(\tau, w, \{p_u\}_{u\in[\kappa]}, \mathcal{B})$ is public; the prover knows W. We write $\mathbf{t}(x) := (t_0(x), \ldots, t_{w-1}(x))$.

1: $\mathcal{P}$ sends the trace polynomials $t_0, \ldots, t_{w-1} \in \mathbb{F}[T]^{<\tau}$.

2: $\mathcal{V}$ samples $\gamma \leftarrow\$ \mathbb{F}$; $\mathcal{P}$ sends the **composition polynomial**

$$\psi := \sum_{u\in[\kappa]} \gamma^u \cdot q_u^{\mathcal{T}} + \sum_{j\in[w]} \gamma^{\kappa+j} \cdot q_j^{\mathcal{B}} \in \mathbb{F}[T]^{<\tau}.$$

3: $\mathcal{V}$ samples $r_0 \leftarrow\$ \mathbb{F}$, queries $t_j(r_0)$, $t_j(\omega_\tau r_0)$ for all $j \in [w]$, and $\psi(r_0)$, and checks

$$\psi(r_0) = \sum_{u\in[\kappa]} \gamma^u \frac{p_u(\mathbf{t}(r_0), \mathbf{t}(\omega_\tau r_0))}{\nu_{\mathcal{T}}(r_0)} + \sum_{j\in[w]} \gamma^{\kappa+j} \frac{t_j(r_0) - \iota_j(r_0)}{\nu_j(r_0)},$$

AIR Poly-IOP: Why it Works

- The right-hand side is computed by $\mathcal{V}$ from the queried values and the public p_u, ι_j, ν_j, $\nu_{\mathcal{T}}$.

AIR Poly-IOP: Why it Works

- The right-hand side is computed by $\mathcal{V}$ from the queried values and the public p_u, ι_j, ν_j, $\nu_{\mathcal{T}}$.
- **Completeness:** for a valid trace, the right-hand side is the evaluation of the polynomial ψ at r_0.

AIR Poly-IOP: Why it Works

- The right-hand side is computed by $\mathcal{V}$ from the queried values and the public p_u, ι_j, ν_j, $\nu_{\mathcal{T}}$.
- **Completeness:** for a valid trace, the right-hand side is the evaluation of the polynomial ψ at r_0.
- **Soundness:** if some constraint fails, some quotient is not a polynomial, and with high probability over γ neither is their combination; no polynomial ψ agrees with it at a random r_0, except with probability about $2\tau/\#\mathbb{F}$ (Schwartz–Zippel).

AIR Poly-IOP: Example

Example

With $\gamma = 2$: $\psi = q_0^{\mathcal{T}} + 2q_1^{\mathcal{T}} + 4q_0^{\mathcal{B}} + 8q_1^{\mathcal{B}} = 4 + 12T + 6T^2 + 5T^3$. For $r_0 = 2$ ($\omega_\tau r_0 = 9$), $\mathcal{V}$ learns $t_0(2) = 12$, $t_0(9) = 4$, $t_1(2) = 16$, $t_1(9) = 11$, $\psi(2) = 7$; it computes $p_0 = 5$, $p_1 = 2$, $\nu_{\mathcal{T}}(2) = 1$ and checks

$$5 + 2 \cdot 2 + 4 \cdot \frac{12 - 1}{2 - 1} + 8 \cdot \frac{16 - 13}{2 - 4} = 7 = \psi(2).$$

AIR Poly-IOP: Example

Example

With $\gamma = 2$: $\psi = q_0^{\mathcal{T}} + 2q_1^{\mathcal{T}} + 4q_0^{\mathcal{B}} + 8q_1^{\mathcal{B}} = 4 + 12T + 6T^2 + 5T^3$. For $r_0 = 2$ ($\omega_\tau r_0 = 9$), $\mathcal{V}$ learns $t_0(2) = 12$, $t_0(9) = 4$, $t_1(2) = 16$, $t_1(9) = 11$, $\psi(2) = 7$; it computes $p_0 = 5$, $p_1 = 2$, $v_{\mathcal{T}}(2) = 1$ and checks

$$5 + 2 \cdot 2 + 4 \cdot \frac{12 - 1}{2 - 1} + 8 \cdot \frac{16 - 13}{2 - 4} = 7 = \psi(2).$$

Question #5

Check that $q_1^{\mathcal{B}} = 15 + 9T + 2T^2$ is consistent with the check above: compute $q_1^{\mathcal{B}}(2)$ and compare it with $(t_1(2) - 13)/(2 - 4)$.

Compiling the AIR Poly-IOP

$w+1$ polynomials of degree $< \tau$, queried at r_0 and $\omega_\tau r_0$: the input of the previous lecture's compilation with $m = w+1$ and $t = 1$.

Compiling the AIR Poly-IOP

$w+1$ polynomials of degree $< \tau$, queried at r_0 and $\omega_\tau r_0$: the input of the previous lecture's compilation with $m = w+1$ and $t = 1$.

1: **Commit.** $\mathcal{P}$ sends $\overline{t_0}, \ldots, \overline{t_{w-1}}$ and $\overline{\psi}$ on Ω of size $n = \tau/\rho$.

Compiling the AIR Poly-IOP

$w+1$ polynomials of degree $< \tau$, queried at r_0 and $\omega_\tau r_0$: the input of the previous lecture's compilation with $m = w+1$ and $t = 1$.

1: **Commit.** $\mathcal{P}$ sends $\overline{t_0}, \ldots, \overline{t_{w-1}}$ and $\overline{\psi}$ on Ω of size $n = \tau/\rho$.

2: **Bind and query.** r_0 is already random, so it doubles as the binding point; we only need $r_0 \notin \Omega$. $\mathcal{P}$ sends the claimed values at r_0 and $\omega_\tau r_0$.

Compiling the AIR Poly-IOP

$w+1$ polynomials of degree $< \tau$, queried at r_0 and $\omega_\tau r_0$: the input of the previous lecture's compilation with $m = w+1$ and $t = 1$.

1: **Commit.** $\mathcal{P}$ sends $\overline{t_0}, \ldots, \overline{t_{w-1}}$ and $\overline{\psi}$ on Ω of size $n = \tau/\rho$.

2: **Bind and query.** r_0 is already random, so it doubles as the binding point; we only need $r_0 \notin \Omega$. $\mathcal{P}$ sends the claimed values at r_0 and $\omega_\tau r_0$.

3: **Quotient and batch.** Virtual quotients $\mathsf{q}_j := (g_j - \iota'_j)/\nu'$ with $\nu' := (T - r_0)(T - \omega_\tau r_0)$, where g_j runs over the committed oracles and ι'_j interpolates the claimed values; batch into $h := \sum_j \alpha^j \mathsf{q}_j$.

Compiling the AIR Poly-IOP

$w+1$ polynomials of degree $< \tau$, queried at r_0 and $\omega_\tau r_0$: the input of the previous lecture's compilation with $m = w+1$ and $t = 1$.

1: **Commit.** $\mathcal{P}$ sends $\overline{t_0}, \ldots, \overline{t_{w-1}}$ and $\overline{\psi}$ on Ω of size $n = \tau/\rho$.
2: **Bind and query.** r_0 is already random, so it doubles as the binding point; we only need $r_0 \notin \Omega$. $\mathcal{P}$ sends the claimed values at r_0 and $\omega_\tau r_0$.
3: **Quotient and batch.** Virtual quotients $\mathsf{q}_j := (g_j - \iota'_j)/\nu'$ with $\nu' := (T - r_0)(T - \omega_\tau r_0)$, where g_j runs over the committed oracles and ι'_j interpolates the claimed values; batch into $h := \sum_j \alpha^j \mathsf{q}_j$.
4: **Test.** Run FRI on h against $\mathrm{RS}[\mathbb{F}, \Omega, \tau - 2]$.

Compiling the AIR Poly-IOP

$w+1$ polynomials of degree $< \tau$, queried at r_0 and $\omega_\tau r_0$: the input of the previous lecture's compilation with $m = w+1$ and $t = 1$.

1: **Commit.** $\mathcal{P}$ sends $\overline{t_0}, \ldots, \overline{t_{w-1}}$ and $\overline{\psi}$ on Ω of size $n = \tau/\rho$.
2: **Bind and query.** r_0 is already random, so it doubles as the binding point; we only need $r_0 \notin \Omega$. $\mathcal{P}$ sends the claimed values at r_0 and $\omega_\tau r_0$.
3: **Quotient and batch.** Virtual quotients $\mathsf{q}_j := (g_j - \iota'_j)/\nu'$ with $\nu' := (T - r_0)(T - \omega_\tau r_0)$, where g_j runs over the committed oracles and ι'_j interpolates the claimed values; batch into $h := \sum_j \alpha^j \mathsf{q}_j$.
4: **Test.** Run FRI on h against $\mathrm{RS}[\mathbb{F}, \Omega, \tau - 2]$.

After BCS, this is a **STARK**: essentially the DEEP-ALI protocol used by deployed STARKs.

The Evaluation Domain

The verifier divides by $v_{\mathcal{T}}$, v_j (vanish on H) and v' (vanishes at r_0, $\omega_\tau r_0$); FRI needs $-x \in \Omega$ for every $x \in \Omega$.

The Evaluation Domain

The verifier divides by $v_{\mathcal{T}}$, v_j (vanish on H) and v' (vanishes at r_0, $\omega_\tau r_0$); FRI needs $-x \in \Omega$ for every $x \in \Omega$.

Remark

Let ω be a primitive nth root of unity with $\omega^{n/\tau} = \omega_\tau$, so $H \subseteq \langle \omega \rangle$, and take the **coset** $\Omega := c \cdot \langle \omega \rangle$ for some $c \notin \langle \omega \rangle$. Then $\Omega \cap H = \emptyset$, $-x \in \Omega$ for every $x \in \Omega$, and squaring maps Ω onto $c^2 \cdot \langle \omega^2 \rangle$.

The Evaluation Domain

The verifier divides by $v_{\mathcal{T}}$, v_j (vanish on H) and v' (vanishes at r_0, $\omega_\tau r_0$); FRI needs $-x \in \Omega$ for every $x \in \Omega$.

Remark

Let ω be a primitive nth root of unity with $\omega^{n/\tau} = \omega_\tau$, so $H \subseteq \langle \omega \rangle$, and take the **coset** $\Omega := c \cdot \langle \omega \rangle$ for some $c \notin \langle \omega \rangle$. Then $\Omega \cap H = \emptyset$, $-x \in \Omega$ for every $x \in \Omega$, and squaring maps Ω onto $c^2 \cdot \langle \omega^2 \rangle$.

Example

$\rho = 1/2$, $n = 8$, $\omega = 9$ of order 8 in $\mathbb{F}_{17}^{\times}$, $\omega^2 = 13 = \omega_\tau$:

$$\langle \omega \rangle = \{1, 9, 13, 15, 16, 8, 4, 2\} \supseteq H.$$

$c = 3 \notin \langle \omega \rangle$, so $\Omega = 3 \cdot \langle \omega \rangle = \{3, 10, 5, 11, 14, 7, 12, 6\}$. Indeed $\Omega \cap H = \emptyset$, and $r_0 = 2$, $\omega_\tau r_0 = 9$ lie outside Ω.

Example: Commit, Quotient, Batch

The prover commits to three codewords:

$x \in \Omega$	3	10	5	11	14	7	12	6
$\overline{t_0}(x)$	2	5	12	1	13	13	11	2
$\overline{t_1}(x)$	4	7	4	6	5	14	3	6
$\overline{\psi}(x)$	8	12	6	3	6	6	13	12

Example: Commit, Quotient, Batch

The prover commits to three codewords:

$x \in \Omega$	3	10	5	11	14	7	12	6
$\overline{t_0}(x)$	2	5	12	1	13	13	11	2
$\overline{t_1}(x)$	4	7	4	6	5	14	3	6
$\overline{\psi}(x)$	8	12	6	3	6	6	13	12

- Besides the values at $r_0 = 2$, $\mathcal{P}$ sends $t_0(9) = 4$, $t_1(9) = 11$, $\psi(9) = 10$.

Example: Commit, Quotient, Batch

The prover commits to three codewords:

$x \in \Omega$	3	10	5	11	14	7	12	6
$\overline{t_0}(x)$	2	5	12	1	13	13	11	2
$\overline{t_1}(x)$	4	7	4	6	5	14	3	6
$\overline{\psi}(x)$	8	12	6	3	6	6	13	12

- Besides the values at $r_0 = 2$, $\mathcal{P}$ sends $t_0(9) = 4$, $t_1(9) = 11$, $\psi(9) = 10$.
- With $\nu' = (T-2)(T-9)$: $\mathsf{q}_0 = 11 + 6T$, $\mathsf{q}_1 = 6 + 2T$, $\mathsf{q}_2 = 10 + 5T$.

Example: Commit, Quotient, Batch

The prover commits to three codewords:

$x \in \Omega$	3	10	5	11	14	7	12	6
$\overline{t_0}(x)$	2	5	12	1	13	13	11	2
$\overline{t_1}(x)$	4	7	4	6	5	14	3	6
$\overline{\psi}(x)$	8	12	6	3	6	6	13	12

- Besides the values at $r_0 = 2$, $\mathcal{P}$ sends $t_0(9) = 4$, $t_1(9) = 11$, $\psi(9) = 10$.
- With $\nu' = (T-2)(T-9)$: $\mathsf{q}_0 = 11 + 6T$, $\mathsf{q}_1 = 6 + 2T$, $\mathsf{q}_2 = 10 + 5T$.
- With $\alpha = 3$: $h = \mathsf{q}_0 + 3\mathsf{q}_1 + 9\mathsf{q}_2 = 6T \in \mathsf{RS}[\mathbb{F}_{17}, \Omega, 2]$.

Example: Commit, Quotient, Batch

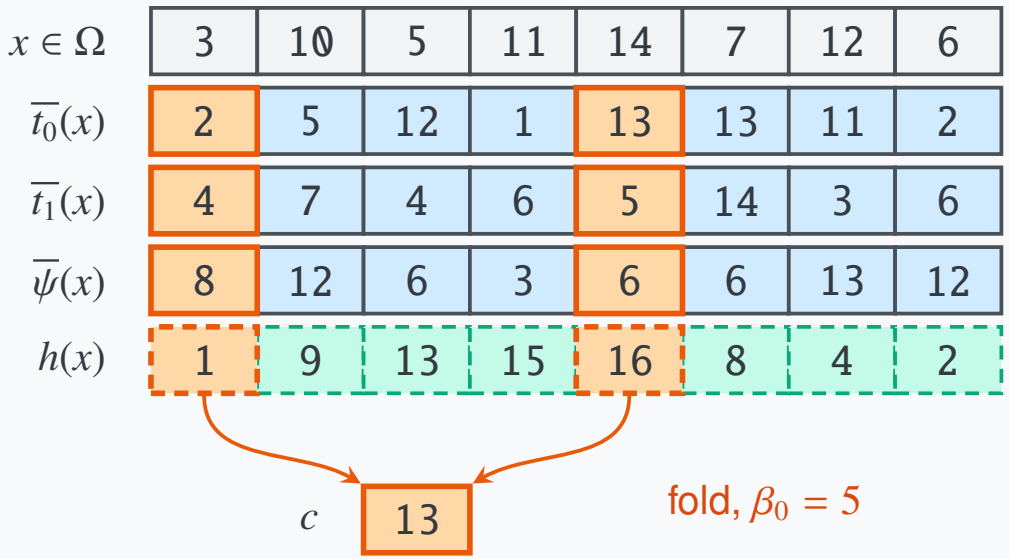
The prover commits to three codewords:

$x \in \Omega$	3	10	5	11	14	7	12	6
$\overline{t_0}(x)$	2	5	12	1	13	13	11	2
$\overline{t_1}(x)$	4	7	4	6	5	14	3	6
$\overline{\psi}(x)$	8	12	6	3	6	6	13	12

- Besides the values at $r_0 = 2$, $\mathcal{P}$ sends $t_0(9) = 4$, $t_1(9) = 11$, $\psi(9) = 10$.
- With $\nu' = (T-2)(T-9)$: $\mathsf{q}_0 = 11 + 6T$, $\mathsf{q}_1 = 6 + 2T$, $\mathsf{q}_2 = 10 + 5T$.
- With $\alpha = 3$: $h = \mathsf{q}_0 + 3\mathsf{q}_1 + 9\mathsf{q}_2 = 6T \in \mathsf{RS}[\mathbb{F}_{17}, \Omega, 2]$.
- FRI needs a single fold: with $\beta_0 = 5$, the fold of $\overline{h}$ is the constant 13 on $\Omega^2 = \{9, 15, 8, 2\}$.

Example: the Verifier

$\mathcal{V}$ samples $x_0 = 3$, so it needs $h(3)$ and $h(-3) = h(14)$, and opens the committed codewords at 3 and 14:



The last row is **not** committed: $\mathcal{V}$ computes it from the claimed values at $r_0 = 2$ and $\omega_\tau r_0 = 9$.

Example: the Verifier

- $v'(3) = (3-2)(3-9) = 11$, and the interpolant of the claimed values of t_0 is $\iota'_0 = 7 + 11T$, so $q_0(3) = (2-6)/11 = 12$.

Example: the Verifier

- $v'(3) = (3-2)(3-9) = 11$, and the interpolant of the claimed values of t_0 is $\iota'_0 = 7 + 11T$, so $q_0(3) = (2-6)/11 = 12$.
- In the same way, $q_1(3) = 12$ and $q_2(3) = 8$, hence

$$h(3) = 12 + 3 \cdot 12 + 9 \cdot 8 = 1.$$

Example: the Verifier

- $v'(3) = (3-2)(3-9) = 11$, and the interpolant of the claimed values of t_0 is $\iota'_0 = 7 + 11T$, so $\mathsf{q}_0(3) = (2-6)/11 = 12$.
- In the same way, $\mathsf{q}_1(3) = 12$ and $\mathsf{q}_2(3) = 8$, hence

$$h(3) = 12 + 3 \cdot 12 + 9 \cdot 8 = 1.$$

- Together with $h(14) = 16$, the round consistency check gives

$$\frac{1+16}{2} + 5 \cdot \frac{1-16}{2 \cdot 3} = 0 + 13 = 13 = c, \quad \text{so } \mathcal{V} \text{ accepts.}$$

Example: the Verifier

- $v'(3) = (3-2)(3-9) = 11$, and the interpolant of the claimed values of t_0 is $\iota'_0 = 7 + 11T$, so $q_0(3) = (2-6)/11 = 12$.
- In the same way, $q_1(3) = 12$ and $q_2(3) = 8$, hence

$$h(3) = 12 + 3 \cdot 12 + 9 \cdot 8 = 1.$$

- Together with $h(14) = 16$, the round consistency check gives

$$\frac{1+16}{2} + 5 \cdot \frac{1-16}{2 \cdot 3} = 0 + 13 = 13 = c, \quad \text{so } \mathcal{V} \text{ accepts.}$$

Toy parameters

Over $\mathbb{F}_{17}$ the soundness error is far from negligible, and the opened values reveal a lot about the trace; real STARKs use large fields, many queries and randomization of the trace.

Example: the Verifier

- $\nu'(3) = (3-2)(3-9) = 11$, and the interpolant of the claimed values of t_0 is $\iota'_0 = 7 + 11T$, so $\mathsf{q}_0(3) = (2-6)/11 = 12$.
- In the same way, $\mathsf{q}_1(3) = 12$ and $\mathsf{q}_2(3) = 8$, hence

$$h(3) = 12 + 3 \cdot 12 + 9 \cdot 8 = 1.$$

- Together with $h(14) = 16$, the round consistency check gives

$$\frac{1+16}{2} + 5 \cdot \frac{1-16}{2 \cdot 3} = 0 + 13 = 13 = c, \quad \text{so } \mathcal{V} \text{ accepts.}$$

Toy parameters

Over $\mathbb{F}_{17}$ the soundness error is far from negligible, and the opened values reveal a lot about the trace; real STARKs use large fields, many queries and randomization of the trace.

Question #6

Compute $h(14)$ from the opened values, as the verifier does.

Overview of the STARK Protocol

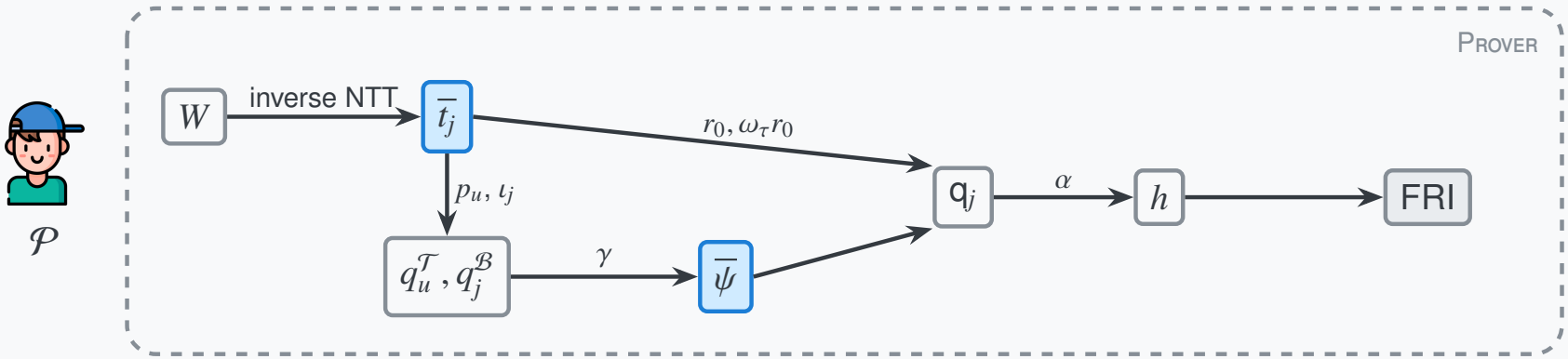


Figure: The prover commits only to $\overline{t_j}$, $\overline{\psi}$ (highlighted) and the FRI layers; q_j and h are virtual.

Overview of the STARK Protocol

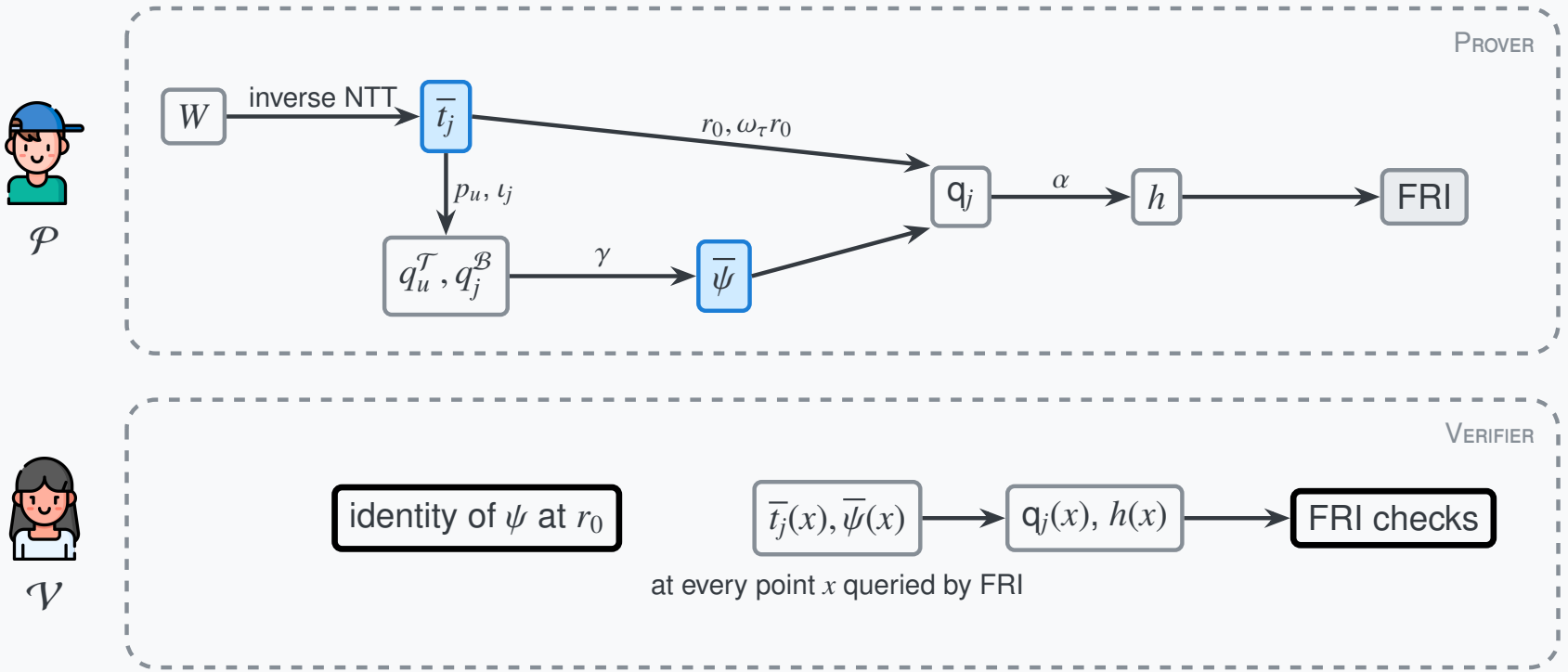


Figure: The prover commits only to $\overline{t_j}$, $\overline{\psi}$ (highlighted) and the FRI layers; q_j and h are virtual.

Overview of the STARK Protocol

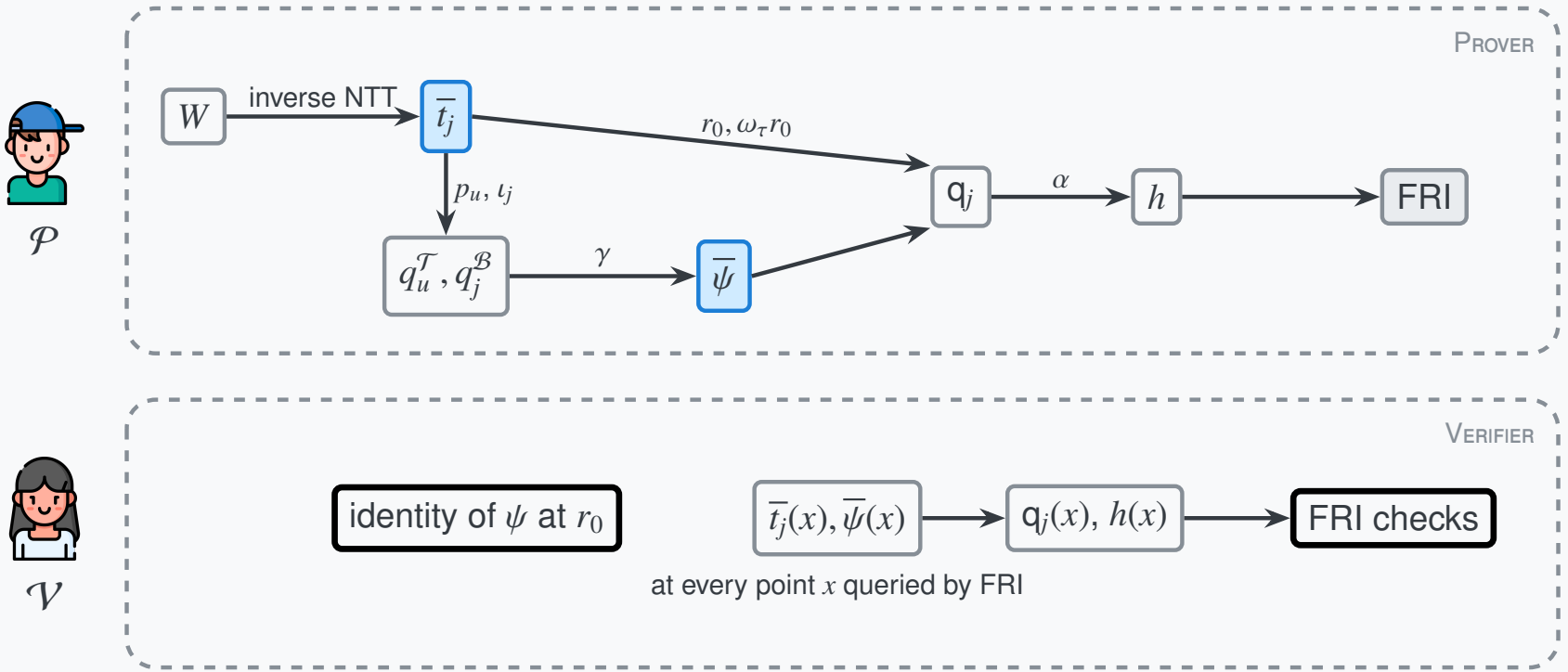


Figure: The prover commits only to $\overline{t_j}$, $\overline{\psi}$ (highlighted) and the FRI layers; q_j and h are virtual.

If the trace is wrong, some quotient is not a polynomial, so the identity of ψ fails at a random r_0; quotienting and FRI bind the values at r_0 to the committed codewords.

Final Remarks

Remark

An alternative avoids the out-of-domain point: commit to the boundary quotients instead of the trace, and run FRI on a random combination of all quotients, each multiplied by T^{k-1-d_q} (**degree correction**). The known soundness bounds are weaker, so deployed STARKs use the protocol above.

Final Remarks

Remark

An alternative avoids the out-of-domain point: commit to the boundary quotients instead of the trace, and run FRI on a random combination of all quotients, each multiplied by T^{k-1-d_q} (**degree correction**). The known soundness bounds are weaker, so deployed STARKs use the protocol above.

Remark

This protocol is **not** zero-knowledge: opened values of t_j and ψ reveal information about the trace. The original construction randomizes the trace polynomials outside of H and masks the input of FRI with a random polynomial. Many applications (e.g., validity rollups) only need succinctness.

Final Remarks

Remark

An alternative avoids the out-of-domain point: commit to the boundary quotients instead of the trace, and run FRI on a random combination of all quotients, each multiplied by T^{k-1-d_q} (**degree correction**). The known soundness bounds are weaker, so deployed STARKs use the protocol above.

Remark

This protocol is **not** zero-knowledge: opened values of t_j and ψ reveal information about the trace. The original construction randomizes the trace polynomials outside of H and masks the input of FRI with a random polynomial. Many applications (e.g., validity rollups) only need succinctness.

Remark

As with FRI, the whole protocol is made non-interactive using the Fiat–Shamir transformation.

Summary

- **FRI**: fold $g_i \mapsto g_{i+1} = \text{Fold}(g_i, \beta_i)$ over $\Omega_i = \Omega^{2^i}$ until a constant; check round consistency along s random query paths.

Summary

- **FRI**: fold $g_i \mapsto g_{i+1} = \text{Fold}(g_i, \beta_i)$ over $\Omega_i = \Omega^{2^i}$ until a constant; check round consistency along s random query paths.
- Folding is **local**: one value of the fold needs only $g_i(\pm x_i)$; $O(n)$ prover, $O(s \log n)$ queries, soundness error $\ell \cdot \varepsilon_\delta + (1 - \delta)^s$.

Summary

- **FRI**: fold $g_i \mapsto g_{i+1} = \text{Fold}(g_i, \beta_i)$ over $\Omega_i = \Omega^{2^i}$ until a constant; check round consistency along s random query paths.
- Folding is **local**: one value of the fold needs only $g_i(\pm x_i)$; $O(n)$ prover, $O(s \log n)$ queries, soundness error $\ell \cdot \varepsilon_\delta + (1 - \delta)^s$.
- **AIR**: execution trace $W \in \mathbb{F}^{\tau \times w}$, transition verification polynomials p_u and boundary conditions $\mathcal{B}$.

Summary

- **FRI**: fold $g_i \mapsto g_{i+1} = \text{Fold}(g_i, \beta_i)$ over $\Omega_i = \Omega^{2^i}$ until a constant; check round consistency along s random query paths.
- Folding is **local**: one value of the fold needs only $g_i(\pm x_i)$; $O(n)$ prover, $O(s \log n)$ queries, soundness error $\ell \cdot \varepsilon_\delta + (1-\delta)^s$.
- **AIR**: execution trace $W \in \mathbb{F}^{\tau \times w}$, transition verification polynomials p_u and boundary conditions $\mathcal{B}$.
- Interpolate the trace over $H = \langle \omega_\tau \rangle$; constraints become **divisibility**: $q_u^{\mathcal{T}} = c_u / \nu_{\mathcal{T}}$ and $q_j^{\mathcal{B}} = (t_j - \iota_j)/\nu_j$ are polynomials.

Summary

- **FRI**: fold $g_i \mapsto g_{i+1} = \text{Fold}(g_i, \beta_i)$ over $\Omega_i = \Omega^{2^i}$ until a constant; check round consistency along s random query paths.
- Folding is **local**: one value of the fold needs only $g_i(\pm x_i)$; $O(n)$ prover, $O(s \log n)$ queries, soundness error $\ell \cdot \varepsilon_\delta + (1-\delta)^s$.
- **AIR**: execution trace $W \in \mathbb{F}^{\tau \times w}$, transition verification polynomials p_u and boundary conditions $\mathcal{B}$.
- Interpolate the trace over $H = \langle \omega_\tau \rangle$; constraints become **divisibility**: $q_u^{\mathcal{T}} = c_u / v_{\mathcal{T}}$ and $q_j^{\mathcal{B}} = (t_j - \iota_j)/v_j$ are polynomials.
- Batch them into ψ, check at a random r_0, compile with quotienting + FRI on a coset Ω, then BCS: a **STARK**.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream